Deterministic s–t and All-Pairs Minimum Cuts with Subquadratic Cut Queries*

Troy Lee †

Abstract

A cut query to an unknown graph names a set of vertices and returns the number of edges leaving it. We give deterministic algorithms for exact minimum cuts between terminals in a simple unweighted graph on n vertices, for one prescribed pair and for all pairs at once. Both use a subquadratic number of cut queries, well below the near-quadratic cost of learning the graph itself.

The first result concerns a prescribed pair of terminals s and t. We find a minimum s–t cut, together with an explicit maximum flow witnessing its optimality, with $O(n^{3/2}\log^3(2n))$ cut queries. The previous best deterministic bound was $\widetilde{O}(n^{5/3})$, and $\widetilde{O}(n^{3/2})$ was known only for randomized algorithms.

The second result handles all pairs at once. With $O(n^{5/3}\log^5(2n))$ cut queries we determine, for every pair of vertices, the value of a minimum cut separating them and a vertex set attaining it. Before, a subquadratic bound was only known in the randomized case.

The single-pair algorithm learns edges of the graph until an explicit subgraph supports a maximum flow, choosing which edges to reveal through a fixed family of canonical terminal cuts. The all-pairs algorithm uses terminal flows to build one partition of the vertices that preserves a minimum cut for every pair, from which the cuts are read off exactly; it outputs cut values and shores, not flows.

1 Introduction

A cut query reveals the number of edges crossing a specified vertex set without identifying those edges. Such access is sufficient for many global minimum-cut algorithms [29, 26, 5, 4, 18], but a terminal constraint changes which structural information is useful. A graph with a very small global cut can still have large edge connectivity between the prescribed terminals. Preserving one global optimum therefore need not preserve a terminal optimum.

We study exact single-pair and all-pairs terminal cuts in simple unweighted undirected graphs. Our approach maintains an explicit subgraph of the unknown input and learns additional edges until it supports a maximum flow. The central issue is deciding which edges to reveal. The witness method of Jiang, Nanongkai, and Sawettamalya [14] associates a potential to a family of deficient cuts and uses a deterministic cut-query routine to find an edge whose endpoints have substantially different potentials. That edge eliminates a useful fraction of the remaining witnesses. The number of witnesses controls how many such discoveries are needed.

*This preliminary draft was written by Codex GPT-6 Astra; see the AI use statement just before the references. The main theorems have all been verified in Lean 4. Appendix A gives the correspondence between all numbered statements in the paper and their Lean counterparts, and notes where they differ. I have not yet reviewed the prose proofs here. Corrections are welcome.

†Centre for Quantum Software and Information, University of Technology Sydney. troyjlee@gmail.com

Here the additional structure is a fixed family of canonical terminal shores. A preliminary degree-peeling procedure makes the graph outside a core explicit. Once the core labels are fixed, an optimal extension through this known part can be chosen permanently. The family remains valid as new core edges are learned. A minimum-degree subgraph of the core then limits the number of its small cuts. This gives the witness bound needed to reach the three-halves exponent deterministically.

For all pairs, discovering a separate original flow for each pair would repeat too much work. Gomory and Hu [11] reduce all pairs to $n-1$ maximum-flow computations, and in unweighted graphs polylogarithmically many suffice [1], but exact maximum flow is the expensive step in the cut-query model [20]. We instead use terminal flows to build one partition that preserves the cuts needed by every pair. The intermediate networks have two known terminal stars with large capacities. Their internal residual geometry still reflects the simple input graph. This distinction allows a deterministic cut-matching construction [22, 21], followed by a flow that removes the effect of its explicitly recorded fake edges, as in the cut-query expander decomposition of Anand, Saranurak, and Wang [4] and the trimming step of Saranurak and Wang [30]. The remaining large part is an induced expander and requires no further recursion. A degree-based shaving step, as in the friendly-cut sparsifiers of Abboud, Krauthgamer, and Trabelsi [2], then makes a sparse cut representation, which the star transform of Kenneth-Mordoch and Krauthgamer [20] recovers exactly. This is the passage from a learned flow for one pair to one learned representation for all pairs.

1.1 Result and model

Let $V = [n]$, where $n \geq 2$, and let $G = (V, E)$ be a simple, loopless, unweighted undirected graph. The terminals $s, t \in V$ are distinct. For a graph L on V, put

$$\delta_L(S) = \{uv \in E(L) : |\{u, v\} \cap S| = 1\}, \qquad c_L(S) = |\delta_L(S)|.$$

In the single-pair proof we orient terminal shores toward t: $t \in S$ and $s \notin S$. Write

$$\nu(L) = \min_{t \in S,\ s \notin S} c_L(S).$$

The complement is the conventional shore containing s, so either orientation gives the requested cut.

The oracle accepts an arbitrary subset $S \subseteq V$ and returns the exact integer $c_G(S)$. There is no adjacency-list oracle and no separate edge-existence oracle. Each answer lies between 0 and $\lfloor n^2/4 \rfloor$, so it has $O(\log(2n))$ bits. Only calls to this oracle are charged. Enumerating exponentially many objects, optimizing on explicit graphs, exact integer and rational arithmetic, and deterministic tie-breaking are free. All uses of probability in the proofs establish finite counting statements or the existence of a fixed combinatorial object; the algorithm uses no random bits.

Theorem 1 (Exact deterministic terminal cuts)**.** *Given the oracle above, there is a deterministic algorithm that outputs $\nu(G)$, a shore attaining it, and an explicit integral maximum s–t flow using*

$$O\Big(n^{3/2} \log^3(2n)\Big)$$

cut queries. The algorithm always succeeds. Its offline computation need not be polynomial.

Theorem 2 (Exact deterministic all-pairs cuts)**.** *There is a deterministic algorithm which, for every simple unweighted undirected graph G on $n \geq 2$ vertices, returns for each distinct ordered pair (s, t) a set $S_{s,t} \subseteq V$ and an integer $v_{s,t}$ satisfying*

$$s \in S_{s,t}, \quad t \notin S_{s,t}, \qquad v_{s,t} = c_G(S_{s,t}) = \min_{s \in T,\ t \notin T} c_G(T).$$

It uses $O(n^{5/3}\log^5(2n))$ cut queries. Its public query budget and adaptive procedure depend on n before the input is chosen; the same cap can be enforced for arbitrary, even inconsistent, replies. Correctness holds for every input in the stated graph class. Offline computation is unrestricted.

The single-pair proof uses the shore containing t; complementation converts between the two conventions. The all-pairs algorithm constructs an explicit auxiliary graph and restricts its minimum shores to original vertices. Auxiliary flows need not lift to original flows. In particular, Theorem 2 makes no claim of outputting an explicit original maximum flow for each pair.

An undirected flow is represented by antisymmetric values g_{uv} on ordered pairs, with $|g_{uv}| \le 1$ on an edge and $g_{uv} = 0$ on a nonedge. Its divergence is the total outgoing value. An s–t flow of value a has divergences a at s, $-a$ at t, and zero elsewhere. Integral flows can be represented by oriented unit edges, and, after cancelling cycles, by edge-disjoint terminal paths. "Explicit" means that the algorithm outputs such a representation on edges whose existence it has learned.

This is an algorithmic discovery theorem. It is distinct from classical query certificate complexity, in which queries may be chosen with full knowledge of the input and must force its optimum in every graph consistent with their answers. The padded witnesses below are internal combinatorial objects, not nondeterministic or quantum certificates. Our algorithm does not use an externally supplied flow certificate.

1.2 Related work and the source of the improvement

Rubinstein, Schramm, and Weinberg [29] introduced cut queries to study exact optimization from aggregate graph information, motivated by value-oracle submodular function minimization, and gave a randomized $\widetilde{O}(n^{5/3})$ cut-query algorithm for the terminal problem. General submodular minimization uses $\widetilde{O}(n^2)$ value queries [25, 13], and the best query lower bounds for it are near-linear [12, 6]. For global minimum cuts, later work gave $\widetilde{O}(n)$-query weighted [26] and $O(n)$-query simple-graph [5] randomized algorithms, and the cut dimension gives deterministic lower bounds [12, 24]. Anand, Saranurak, and Wang [4] obtained a deterministic $\widetilde{O}(n^{5/3})$ terminal bound using Dinic's blocking flows [7], and a deterministic $\widetilde{O}(n^{5/3})$ global minimum cut. The version of Jiang, Nanongkai, and Sawettamalya used here, arXiv:2510.18274v1 [14], gives a randomized $\widetilde{O}(n^{8/5})$ terminal algorithm and develops the padded-witness and long-edge arguments on which we build. Its deterministic $\widetilde{O}(n^{11/7})$ result concerns two-party communication; it is not a deterministic cut-query bound. Kenneth-Mordoch and Krauthgamer gave the first non-trivial cut-query algorithm for all pairs, with $\widetilde{O}(n^{7/4})$ randomized queries [19], and subsequently [20] a randomized $\widetilde{O}(n^{3/2})$ cut-query algorithm for all pairs in unweighted undirected graphs, hence also for a prescribed pair. They also show that every randomized algorithm that recovers all edges of a maximum s–t flow needs $\Omega(n^{3/2}/\log n)$ cut queries [20, Lemma 1.8]; this applies to the flow output of Theorem 1. Kenneth-Mordoch [18] recently gave a deterministic $\widetilde{O}(n^{3/2})$ global minimum-cut algorithm, built on a deterministic maximum-flow routine for terminal cuts of value at most the minimum degree, and lists a deterministic $o(n^{5/3})$ bound for minimum s–t cut as open. Outside the cut-query model, Gomory–Hu trees reduce to polylogarithmically many maximum flows in unweighted graphs [1] and can be computed deterministically in almost-linear time [3].

The distinction in the present derivation is the canonical-family count and its preservation during witness elimination. From JNS we use the padded witnesses, their invariant, and the killing rule (Definitions 3.6 and 3.8 and Invariant 3.7), the marginal potential and the witness-loss bound (Section 5.2 and Claims 5.3–5.5), the existence of a long unknown edge (Claim 5.6), the phase loop and its halving schedule (Claim 5.1 and Corollary 5.2), the acyclic-flow support bound (Lemma 2.2), and the deterministic query primitive (Claim 5.7 and Lemma 5.8). The support bound is the

flow-cover lemma of Rubinstein, Schramm, and Weinberg [29]. Every numbered JNS reference in this paper means the arXiv v1 just specified. We state their precise roles and supply the support proof and the relevant long-edge construction for completeness. We make no assertion of an optimal query bound or of priority over all other versions or subsequent literature.

The counting argument uses the classical spanning-tree packing theorem of Tutte [31] and Nash-Williams [27]; see Kaiser [15] for a short proof. Its use to count small cuts follows Karger's tree-packing bound on the number of near-minimum cuts [16]. The companion manuscript on weighted global cuts [23] also uses tree packing, but none of its algorithmic or flow-certificate results is a dependency here.

The all-pairs construction combines existing ingredients. The cut-matching game is due to Khandekar, Rao, and Vazirani [22]; we use the deterministic entropy cut player of Khandekar, Khot, Orecchia, and Vishnoi (KKOV) [21]. Anand, Saranurak, and Wang [4, Appendix A] already implement a cut-matching expander decomposition with cut queries: the KKOV cut player costs no queries, the matching player is a cut-query maximum flow that routes almost perfect b-matchings, fake edges complete them, and the fake edges are then removed by the expander pruning of Saranurak and Wang [30]. Our flows use the blocking-flow framework of [4]. Our pruning flow is the trimming step of [30], solved by one exact flow, and the recursion and the boundary charging follow the expander decomposition of [30]. The shaving step and its analysis are those of the friendly-cut sparsifiers of Abboud, Krauthgamer, and Trabelsi [2], with the friendliness parameters of [20]; Kenneth-Mordoch and Krauthgamer [19] build such sparsifiers with randomized cut queries. The structural lemma and the star representation are those of [20]. We give the precise cut-player normalization and full proofs of the heavy-star, pruning, decomposition, and recovery claims used here. In the friendly-cut argument, the minimal shore is chosen on the unfriendly terminal's side, and degree-two vertices are retained in the proof. What is specific here is the heavy-star flow bound linear in the part size (Theorem 19), which pays for the matching and trimming flows with cut queries, and the combination of the resulting deterministic decomposition with friendly-cut recovery. These ingredients establish the deterministic five-thirds bound; the randomized three-halves all-pairs bound belongs to Kenneth-Mordoch and Krauthgamer. No general deterministic three-halves all-pairs bound is asserted, and no randomized Gomory–Hu reduction of Probst Gutenberg, Kyng, Yuan, and Yuan [28] is assumed.

1.3 Proof overview

Choose an integer $2 \le D \le n$. In $O(nD \log(2n))$ queries we learn a graph K containing every edge incident to a vertex outside a core C, and a subgraph $J \subseteq G[C]$ with minimum degree at least D. For every core labeling x, let S_x be a terminal shore that extends x and minimizes the known K-contribution, with ties broken permanently. The family $\mathcal{A} = \{S_x\}$ contains a minimum terminal shore of every graph $K \cup L$ with $J \subseteq L \subseteq G[C]$.

A simple minimum-degree-D graph on at most n vertices has at most $(2n)^{O((n+b)/D)}$ binary labelings whose cuts have size at most b. For $b \le n$, this gives only $(2n)^{O(n/D)}$ possible canonical shores. Simplicity is essential here: a small vertex set cannot use parallel edges or large capacities to supply all of its degree.

Suppose an explicit learned graph $H = K \cup L$ has $\nu(H) \ge f - k$, where f is a proposed target. Put $k' = \lfloor k/2 \rfloor$. A witness consists of a canonical shore with at most $f - k' - 1$ learned crossing edges, padded to exactly that many crossing pairs. Since fewer than k pairs are padding, the family satisfies

$$\log \max\{1, |\mathcal{W}(H)|\} = O((n/D + k) \log(2n)) . \tag{1}$$

The canonical shore associated with a core labeling is fixed, so adding a learned edge can only remove witnesses.

The marginals of a uniform padded witness define a potential in $[0, 1]$ with terminal values zero and one. If $\nu(G) \geq f$, an augmentation using only $O(n\sqrt{k})$ new edges shows that one unknown edge has potential length $\Omega(\sqrt{k}/n)$. The JNS routine finds an edge of comparable length in $O(\log(2n))$ queries. It kills an $\Omega(\sqrt{k}/n)$ fraction of the witnesses. This gives the phase bound

$$O\left(\left(\frac{n^2}{D\sqrt{k}} + n\sqrt{k}\right)\log^2(2n)\right). \tag{2}$$

Failure to find a long edge correctly refutes f, so no feasibility promise is needed. Summing over geometrically decreasing deficits and then binary searching the target gives Theorem 1.

For all pairs, pad the graph by known triangles, put $N = 3n$, and let $t = \lceil N^{1/3} \rceil$, $b = 2t$, and $R = O(\log(2N))$ be a common cut-player round bound. At a part of size q, each matching or pruning flow costs $O(qRt^2 \log(2N))$ queries. A failed matching gives two smaller active parts; a completed game gives a fake multigraph whose removal leaves a large $b/(4R)$-expander. There are $O(R \log(2N))$ active levels, with total part size at most N at each level. Including all R matching calls and the pruning call per node gives $O(Nt^2R^3 \log^2(2N)) = O(n^{5/3} \log^5(2n))$ queries. The technical proofs are in Sections 12 to 15; Section 9 explains how they fit together.

2 Exact query primitives and the limit of global information

2.1 Bipartite counts and edge discovery

For disjoint vertex sets A, B, let $e_L(A, B)$ be the number of edges between them. The identity

$$2e_L(A, B) = c_L(A) + c_L(B) - c_L(A \cup B) \tag{3}$$

simulates a bipartite edge-count query using at most three cut queries [29, 4]. In particular, for $v \notin T$, the number of neighbors of v in T is

$$e_G(\{v\}, T) = \frac{c_G(\{v\}) + c_G(T) - c_G(T \cup \{v\})}{2}.$$

These are exact integer identities; no approximation or rounding is involved. The next lemma is the standard binary-search edge discovery of Rubinstein, Schramm, and Weinberg [29], in the form of the find-edge query of [14, Proposition 2.10].

Lemma 3 (Finding an edge)**.** *For disjoint A, B, cut access to a simple graph L determines whether $e_L(A, B) = 0$ in $O(1)$ queries. If it is positive, an edge between A and B can be found in $O(\log(2n))$ further queries. All neighbors of a vertex in a specified set can be learned in $O((d + 1)\log(2n))$ queries, where d is their number.*

Proof. Use Equation (3) for the zero test. If the count is positive, partition A into two nearly equal parts and count the edges from one part to B. Retain a part with positive count. Repeating leaves one vertex in A. Bisect B in the same way to find its neighbor. There are at most $2\lceil \log_2 n \rceil$ bisections. For the last assertion, remove every discovered neighbor from the candidate set and repeat until its count is zero. Alternatively, when the initial count is known, stop after that many discoveries. □

If $H \subseteq G$ is explicit, the graph $R = G \setminus H$ has the simulated cut oracle

$$c_R(S) = c_G(S) - c_H(S). \tag{4}$$

Thus each cut query to R costs one query to G, as in JNS [14, Section 3]. The subtraction is valid because H consists only of actual unit edges; it will be used both for discovering new edges and for extracting an optimum shore.

2.2 Why a weak global skeleton does not suffice

For comparison, a weak global skeleton may require only $\lambda(B) \geq \lambda(G)$ and a global optimum S^* of G with $c_B(S^*) \leq \alpha\lambda(G)$, where λ denotes global minimum-cut value. Such a guarantee does not approximate a prescribed terminal cut, even if every edge of B is an edge of G.

Proposition 4. *For every $M \geq 1$ there are a simple unit graph G and a weighted graph B supported on $E(G)$ such that*

$$\lambda(B) = \lambda(G) = c_B(\{x\}) = c_G(\{x\}) = 1, \qquad \nu(G) = 2, \qquad \nu(B) = M.$$

Proof. Let G be the triangle on s, a, t together with the pendant edge sx. Let B be the path x, s, a, t, assigning weights $1, M, M$ in that order. Both global minima equal one and are attained at $\{x\}$. The two edge-disjoint paths s, t and s, a, t show that $\nu(G) = 2$. In B, the terminal path has two edges of weight M; any terminal cut crosses at least one of them and a shore avoiding the pendant edge attains M. □

Here support containment does not mean capacity domination: B may increase the capacities of retained edges, as a weak skeleton is allowed to do. The example disproves an implication from this weak global guarantee to terminal approximation. It does not rule out stronger sparsification guarantees. Our learned graphs instead remain unit subgraphs of G throughout.

3 Learning a core and fixing canonical extensions

Lemma 5 (Preprocessing)**.** *Fix $2 \leq D \leq n$. With $O(nD \log(2n))$ cut queries one can construct a set $C \subseteq V$ and explicit edge-disjoint graphs $K, J \subseteq G$ with the following properties:*

(i) K contains exactly the edges with at least one endpoint outside C, and $|E(K)| < nD$;

(ii) if $C \neq \varnothing$, then $J \subseteq G[C]$, $\delta(J) \geq D$, and $|E(J)| \leq |C|D$;

(iii) every edge not in $K \cup J$ has both endpoints in C.

If $C = \varnothing$, the entire graph is explicit.

Proof. Query each degree $d_G(v) = c_G(\{v\})$. Initially let $C = V$ and keep its current induced degrees. If some vertex $v \in C$ has induced degree less than D, learn its remaining neighbors in C with Lemma 3, remove v, and decrement the degrees of precisely those neighbors. Every edge from a still-active vertex to an earlier removed vertex was learned at that earlier removal. Hence the maintained induced degrees are exact, without further degree queries. Use vertex order whenever several removals are possible.

Each removed vertex exposes fewer than D edges and costs $O(D \log(2n))$ queries, including any empty-row test. An edge outside the final induced core is learned when its first endpoint is removed,

and is counted just once. This proves the claim about K and the query bound for peeling. The final induced graph $G[C]$, if nonempty, has minimum degree at least D.

For each $v \in C$, learn D distinct neighbors inside C. Reuse already discovered incidences and subtract them when searching a row. The remaining count and Lemma 3 allow this in $O(D \log(2n))$ queries per vertex. Let J be the union of the chosen edges. Each vertex has at least its D chosen incidences in J, and the number of distinct edges in the union is at most $|C|D$. Since K has no edge entirely inside C, its edge set is disjoint from $E(J)$. □

If $C = \varnothing$, solve the fully learned graph offline. Otherwise assume $C \neq \varnothing$ and set $H_0 = K \cup J$. The algorithm subsequently maintains

$$H = K \cup L, \qquad J \subseteq L \subseteq G[C]. \tag{5}$$

No further edge incident to a peeled vertex is unknown.

A labeling $x : C \to \{0, 1\}$ is admissible if it assigns zero to s when $s \in C$ and one to t when $t \in C$. For every admissible x, choose a terminal shore S_x with $\mathbf{1}[v \in S_x] = x(v)$ for $v \in C$ that minimizes $c_K(S_x)$. Break ties by the fixed order on all vertex subsets. Put

$$\mathcal{A} = \{S_x : x \text{ admissible}\}. \tag{6}$$

This family is computed from explicit data, possibly by exhaustive search, before any witness phase begins.

Lemma 6 (Fixed canonical cuts)**.** *For every $J \subseteq L \subseteq G[C]$,*

$$\nu(K \cup L) = \min_{S \in \mathcal{A}} c_{K \cup L}(S). \tag{7}$$

The same identity holds for G. The family $\mathcal{A}$ is unchanged by all subsequent edge discoveries.

Proof. Fix a core labeling x. The number of crossing edges of L is the same for every extension of x, because both endpoints of each edge of L are in C. Since K and L are edge-disjoint, minimizing the full cut over these extensions is exactly minimizing the K-contribution. Its chosen minimizer is S_x. Minimizing over admissible x proves Equation (7). Taking $L = G[C]$ gives the assertion for G. The rule defining S_x refers only to K, C and the terminal labels, so discovering further core edges does not change it. □

The argument includes the cases in which one or both terminals were peeled. Their labels are imposed in the extension problem rather than on the core. It also explains why the tie-breaking rule is fixed: reselecting arbitrary representatives during a phase could introduce new witnesses and invalidate the elimination argument.

4 Counting cuts in a simple minimum-degree graph

The required count accommodates disconnected graphs. A lower bound on minimum degree alone does not imply a lower bound on global edge connectivity, so a direct near-minimum-cut count is insufficient. We first split the graph into a controlled number of highly connected parts, keeping track of small parts by simplicity.

Lemma 7 (Small-cut count)**.** *Let J be a simple unweighted graph on $1 \le N \le n$ vertices, with minimum degree at least $D \ge 2$. For an integer $0 \le b \le n$, the number of binary vertex labelings whose cuts have size at most b is at most*

$$(2n)^{O((N+b)/D)}. \tag{8}$$

Both a labeling and its complement are counted.

Proof. Let $\tau = D/8$. Start with the single part $V(J)$ and repeatedly split a nonsingleton part whenever its induced graph has a nontrivial cut of size less than τ. At termination each part is either a singleton or has internal edge connectivity at least τ. If there are $q > 1$ final parts, the total number M of edges between parts obeys

$$M < \tau(q-1). \tag{9}$$

Indeed, there are $q-1$ splits, and each final interpart edge is counted at the unique split that first separates its endpoints. For $q = 1$ we have $M = 0$.

Call a part small if its size is less than $D/2$. Every vertex in a small part has more than $D/2$ neighbors outside that part, since J is simple and has minimum degree at least D. If N_s is the number of vertices in small parts and q_s their number, then for $q > 1$,

$$\frac{D}{2}N_s \le 2M < \frac{D}{4}(q-1), \qquad q_s \le N_s < \frac{q-1}{2}.$$

Thus at least $q/2$ parts have size at least $D/2$, giving

$$q \le \frac{4N}{D}. \tag{10}$$

This is also true for $q = 1$, since $N \ge D + 1$.

Now consider a nonsingleton final part P, whose edge connectivity λ_P is at least τ. The tree-packing theorem of Tutte [31] and Nash-Williams [27] gives a fractional spanning-tree packing of mass at least $\lambda_P/2$. For precision, double every edge of $J[P]$. For a partition into $r \ge 2$ nonempty sets, there are at least $r\lambda_P$ crossing edges in this doubled multigraph, by summing the r boundary sizes and dividing by two. This is at least $\lambda_P(r-1)$, so the Tutte–Nash-Williams theorem supplies λ_P edge-disjoint spanning trees. Projecting them back and assigning each weight $1/2$ gives the stated fractional packing in $J[P]$. The normalized distribution on its trees therefore satisfies

$$\mathbb{E}_T|\delta_T(A)| \le \frac{2c_{J[P]}(A)}{\lambda_P} \le \frac{2c_{J[P]}(A)}{\tau}. \tag{11}$$

Only this classical packing theorem is used as an external fact.

The following count applies to each part Karger's tree-packing argument for near-minimum cuts [16]. Fix an integer budget $a \ge \tau$. Every cut of size at most a crosses at most $4a/\tau$ edges of a sampled tree with probability at least $1/2$, by Markov's inequality. For any fixed tree on P, a set of i crossing tree edges and the label of one root determine all labels. Consequently the number of labelings that cross at most $4a/\tau$ tree edges is at most

$$2\sum_{i=0}^{\min\{|P|-1,\lfloor 4a/\tau\rfloor\}} \binom{|P|-1}{i} \le (2n)^{O(1+a/\tau)}.$$

Sum the at-least-1/2 probabilities over the labelings under consideration and interchange the sum and expectation. This bounds their number by twice the displayed expression. If $0 \le a < \tau$, only

the two constant labelings are possible. A singleton also has only two labelings. We have therefore obtained the uniform bound $(2n)^{O(1+a/\tau)}$ for every part and every nonnegative budget.

For a labeling of all of J, let a_i count its crossing edges internal to part i. A cut of size at most b satisfies $\sum_i a_i \le b$. Since $b \le n$, there are at most $(n+1)^q$ possible integer budget vectors. For any fixed vector, the number of labelings is at most

$$\prod_{i=1}^{q} (2n)^{O(1+a_i/\tau)} = (2n)^{O(q+b/\tau)}.$$

Ignoring the interpart edges only enlarges this count. Multiplying by the number of vectors and using Equation (10) and $\tau = D/8$ proves Equation (8). □

Corollary 8. *For every learned graph H of the form Equation (5) and $0 \le b \le n$, there are at most $(2n)^{O(n/D)}$ shores $S \in \mathcal{A}$ with $c_H(S) \le b$.*

Proof. The restriction of such a shore to C has cut size at most b in $J \subseteq H$. Apply Lemma 7 to J, with $N = |C| \le n$. Each core labeling has exactly one canonical extension. □

5 Restricted padded witnesses

Fix integers $1 \le f \le n-1$ and $1 \le k \le f$, and suppose that the current learned graph satisfies

$$\nu(H) \ge f - k. \tag{12}$$

The target f is not assumed feasible in G. Define

$$k' = \lfloor k/2 \rfloor, \qquad b = f - k' - 1. \tag{13}$$

Here $0 \le b \le n-2$.

For a terminal shore S, write $\Pi(S)$ for its set of possible crossing unordered pairs. When convenient, represent each pair once with its endpoint in S first; then $|\Pi(S)| = |S|(n-|S|)$. In particular $\Pi(S)$ includes nonedges.

Definition 9 (Padded witness)**.** A witness for the phase (f,k) in H is a pair (S,Y) such that

$$S \in \mathcal{A}, \qquad c_H(S) \le b, \qquad \delta_H(S) \subseteq Y \subseteq \Pi(S), \qquad |Y| = b. \tag{14}$$

Let $\mathcal{W}(H)$ be the finite family of all such pairs.

Padding by possible pairs is essential. It neither asserts that a padding pair is an edge of G nor adds that pair to the learned graph. The construction is the padded-witness method of JNS [14, Definition 3.6], restricted to the fixed family $\mathcal{A}$. Parts (i) and (ii) of the next lemma are the witness invariant and the killing rule of [14, Invariant 3.7 and Definition 3.8] for this restricted family.

Lemma 10 (Witness invariants and count)**.** *During a phase with fixed parameters (f,k):*

(i) $\mathcal{W}(H) = \varnothing$ if and only if $\nu(H) \ge f - k'$;

(ii) adding an actual edge $e \in E(G) \setminus E(H)$ removes exactly the witnesses for which $e \in \Pi(S) \setminus Y$, and creates no witnesses;

(iii)

$$\log \max\{1, |\mathcal{W}(H)|\} = O((n/D + k)\log(2n)). \tag{15}$$

Proof. Every terminal shore is nonempty and proper, and hence has at least $n-1$ possible crossing pairs. Since $b \le n-2$, every shore with $c_H(S) \le b$ can be padded as in Equation (14). By Lemma 6, the family is empty precisely when all terminal cuts in H have size greater than b. Their sizes are integers, so this is equivalent to $\nu(H) \ge b+1 = f-k'$.

An unknown edge has both endpoints in C, so adding it preserves Equation (5) and leaves $\mathcal{A}$ fixed. A witness remains a witness exactly when its fixed set Y contains every crossing edge of the larger learned graph. This proves the second assertion, including the case where the cut ceases to have size at most b.

By Corollary 8, at most $(2n)^{O(n/D)}$ shores can occur. For a fixed one, let $h = c_H(S)$. Its padding size satisfies

$$0 \le b - h \le (f-k'-1) - (f-k) = k - k' - 1 < k. \tag{16}$$

The number of choices of padding is at most n^{2k}, since it is a subset of fewer than n^2 pairs with a fixed size less than k. Multiplication proves Equation (15). □

5.1 Exact potentials and the padding distribution

As in [14, Section 5.2], when $\mathcal{W}(H) \neq \varnothing$, choose (S, Y) uniformly from this family for the purpose of defining

$$z_v = \mathbf{1}[v \in S], \qquad \phi_v = \mathbb{E} z_v. \tag{17}$$

Thus $0 \le \phi_v \le 1$, $\phi_s = 0$, and $\phi_t = 1$. The algorithm computes these expectations exactly from explicit data; it does not draw a witness at random.

For a fixed shore, put $M_S = |S|(n-|S|)$ and $h_S = c_H(S)$. Its multiplicity in the uniform padded-witness distribution is

$$w_H(S) = \binom{M_S - h_S}{b - h_S}. \tag{18}$$

Consequently one may compute the marginals as

$$\phi_v = \frac{\displaystyle\sum_{S \in \mathcal{A}:\ h_S \le b} w_H(S)\mathbf{1}[v \in S]}{\displaystyle\sum_{S \in \mathcal{A}:\ h_S \le b} w_H(S)}. \tag{19}$$

This is generally not the uniform distribution on canonical shores. The denominator is $|\mathcal{W}(H)|$, so Equation (15) bounds its bit length by $O((n/D+k)\log(2n))$ up to an additive constant. No real-number oracle or unbounded-precision oracle response is hidden in this step. The number of summands and the time to obtain them may nevertheless be exponential.

The next lemma is [14, Claims 5.3–5.5] for the restricted family, with the same proof.

Lemma 11 (Witness loss from an edge)**.** *For every $uv \in E(G) \setminus E(H)$, adding uv kills at least*

$$\frac{k'+1}{k}|\phi_u - \phi_v| \tag{20}$$

of the current witnesses, as a fraction of $|\mathcal{W}(H)|$.

Proof. By Jensen's inequality,

$$\Pr(z_u \neq z_v) = \mathbb{E}|z_u - z_v| \ge |\phi_u - \phi_v|. \tag{21}$$

Condition on a particular shore S separating the endpoints, with $h = c_H(S)$ and $M = |\Pi(S)|$. Conditional on S, the padding is a uniform $(b-h)$-subset of the $M-h$ pairs outside $\delta_H(S)$. Since $uv \notin H$, the chance it is included is $(b-h)/(M-h)$. The inequalities $M \geq n-1 \geq f$ and $f-k \leq h \leq b < f$ give

$$\Pr(uv \in Y \mid S) = \frac{b-h}{M-h} \leq \frac{b-h}{f-h} = 1 - \frac{k'+1}{f-h} \leq 1 - \frac{k'+1}{k}. \tag{22}$$

If the endpoints are separated and the pair is not in Y, the witness is killed by Lemma 10. Average Equation (22) over the actual shore distribution and then use Equation (21). Uniformity over shores was never assumed. □

6 A short augmentation exposes a long edge

We next establish the structural reason why a feasible target forces an unknown edge of substantial potential length. The argument uses integrality twice: to bound the support of an augmentation and to obtain f edge-disjoint paths in a unit graph.

The next bound is the flow-cover lemma of Rubinstein, Schramm, and Weinberg [29], which JNS restate for unit capacities; the proof below follows theirs. Galil and Yu [10] showed that a simple graph has a maximum flow using $O(n\sqrt{v})$ edges, where v is its value.

Lemma 12 (Acyclic integral flow support; JNS Lemma 2.2)**.** *Let a be a nonnegative integral flow of value $r \geq 1$ on a directed graph with n vertices, after aggregation of parallel arcs. If its positive support is acyclic and its only nonzero divergences are at s, t, the number m of positive arcs satisfies*

$$m \leq \sqrt{2}\, n\sqrt{r}.$$

Proof. Let $p : V \to [n]$ be a topological ordering of the positive support and put $\ell(uv) = p(v) - p(u) > 0$ for a positive arc. Flow conservation gives

$$\sum_{uv} a_{uv}\ell(uv) = -\sum_{v} p(v)\,\mathrm{div}(a)_v = r(p(t) - p(s)) \leq nr. \tag{23}$$

Every positive flow value is an integer at least one. Moreover, for a vertex with d_v distinct out-neighbors, its outgoing arc lengths are distinct positive integers. Hence

$$\sum_{uv} a_{uv}\ell(uv) \geq \sum_{v} \frac{d_v(d_v+1)}{2} \geq \frac{1}{2}\sum_{v} d_v^2 \geq \frac{m^2}{2n}.$$

Together with Equation (23) this proves the bound. □

The next two lemmas are [14, Claim 5.6] and the construction of the set Q in its proof, with explicit constants.

Lemma 13 (Sparse augmentation)**.** *Suppose $H \subseteq G$ is explicit, $\nu(H) \geq f-k$, and $\nu(G) \geq f > \nu(H)$. There is a set $Q \subseteq E(G) \setminus E(H)$ such that*

$$|Q| \leq \sqrt{2}\, n\sqrt{k}, \qquad \nu(H \cup Q) \geq f. \tag{24}$$

This is an existence statement; the algorithm does not learn Q.

Proof. Compute an integral maximum flow g in H and extend it by zero to G. Let $a_0 = \nu(H)$ and $r = f - a_0$, so $1 \le r \le k$. In the residual network of g in G, an original edge uv has residual capacity $1 - g_{uv}$ from u to v and $1 + g_{uv}$ in the reverse direction. These capacities are integers in $\{0, 1, 2\}$. Max-flow/min-cut and the augmentation property [9] give an integral residual flow of value r. Delete circulations until its positive support is acyclic. This preserves its value and feasibility.

The residual graph has at most one aggregated arc in each direction on an original pair. Lemma 12 therefore bounds the number of positive residual arcs by $\sqrt{2}\, n\sqrt{r}$. Let Q consist of the underlying original edges outside H that occur in that support. Adding the augmentation to g, with opposite directions combined antisymmetrically, gives a feasible undirected flow of value f supported in $H \cup Q$. This proves both conclusions. □

Lemma 14 (A long unknown edge)**.** *Suppose $\mathcal{W}(H) \neq \varnothing$ and $\nu(G) \ge f$. For the potential in Equation (17), some $uv \in E(G) \setminus E(H)$ satisfies*

$$|\phi_u - \phi_v| \ge \delta_k, \qquad \delta_k := \frac{k' + 1}{2n\lceil\sqrt{k}\rceil} \ge \frac{\sqrt{k}}{8n}. \tag{25}$$

Proof. Every witness shore has at most b crossing learned edges. Jensen's inequality, followed by linearity of expectation, gives

$$\sum_{uv \in E(H)} |\phi_u - \phi_v| \le \sum_{uv \in E(H)} \mathbb{E}|z_u - z_v| = \mathbb{E}c_H(S) \le b. \tag{26}$$

Since the witness family is nonempty, Lemma 10 gives $\nu(H) \le b < f$. Let Q be supplied by Lemma 13. The graph $H \cup Q$ is unit-capacitated and has a flow of value at least f, so it contains f edge-disjoint s–t paths. The sum of absolute potential differences along each such path is at least $|\phi_t - \phi_s| = 1$. Thus

$$\sum_{uv \in E(H) \cup Q} |\phi_u - \phi_v| \ge f.$$

Subtract Equation (26) to obtain

$$\sum_{uv \in Q} |\phi_u - \phi_v| \ge f - b = k' + 1. \tag{27}$$

Averaging gives an edge of length at least $(k'+1)/(\sqrt{2}\, n\sqrt{k})$, which is at least δ_k. Finally, $k'+1 \ge k/2$ and $\lceil\sqrt{k}\rceil \le 2\sqrt{k}$ prove the last inequality in Equation (25). □

The sparse augmentation is used only to prove that an edge exists. It is not an instruction to reconstruct a residual network. The actual edge is discovered by the following cut-query primitive.

Lemma 15 (Deterministic long-edge search; JNS Lemma 5.8)**.** *Let R be an undirected graph on V, accessible through exact cut queries. Given explicit rational potentials $\phi : V \to [0, 1]$ and a rational threshold $0 < \delta < 1$, there is a deterministic algorithm using $O(\log(n/\delta))$ queries that returns either an edge $uv \in E(R)$ with $|\phi_u - \phi_v| \ge \delta/2$, or* Fail*. A failure certifies that every edge of R has length strictly less than δ.*

This is the interface of JNS Lemma 5.8 in the cited version. Section 11 gives its construction, including exact query simulation, deterministic selection of separating assignments, and the endpoint convention for potential value one. Its use of a probabilistic existence proof does not make the routine randomized. We apply it to $R = G \setminus H$, using Equation (4).

7 A phase without a feasibility promise

The phase is the loop in the proof of [14, Claim 5.1], run on the restricted witnesses. JNS assume the promise $\nu(G) \geq f$; here a failed search refutes the target instead.

Theorem 16 (Deficit reduction or target refutation)**.** *Let H have the form Equation (5), and suppose $1 \leq k \leq f \leq n-1$ and $\nu(H) \geq f-k$. There is a deterministic phase using*

$$O\left(\left(\frac{n^2}{D\sqrt{k}} + n\sqrt{k}\right)\log^2(2n)\right) \tag{28}$$

cut queries that either produces an explicit $H' \supseteq H$ of the same form with $\nu(H') \geq f - \lfloor k/2 \rfloor$, or correctly concludes that $\nu(G) < f$.

Proof. Fix $k' = \lfloor k/2 \rfloor$ and $b = f - k' - 1$ for the entire phase. Repeat the following steps:

1. Compute $\mathcal{W}(H)$ from explicit information. If it is empty, return H with success.
2. Compute ϕ by Equation (19). Invoke FindLongEdge on $G \setminus H$ with threshold δ_k from Equation (25).
3. If the routine fails, return the conclusion $\nu(G) < f$. Otherwise insert its returned edge in H and continue.

Adding edges preserves $\nu(H) \geq f-k$ and the learned-graph form. An empty witness family certifies the required new threshold by Lemma 10. On failure, Lemma 15 says no unknown edge has length at least δ_k; the contrapositive of Lemma 14 proves $\nu(G) < f$. This conclusion does not require an initial promise that f is feasible.

Every successful call returns an edge of length at least $\delta_k/2$. By Lemma 11 and Equation (25), it kills a fraction at least

$$\frac{k'+1}{k}\frac{\delta_k}{2} \geq \frac{\sqrt{k}}{32n} \tag{29}$$

of the witnesses. This multiplicative decrease holds for feasible and infeasible targets alike. Witnesses are never added during the phase. If W_0 is its initial nonzero size and $\alpha = \sqrt{k}/(32n)$, after m successful insertions the size is at most $W_0(1-\alpha)^m \leq W_0 e^{-\alpha m}$. Once this quantity is below one, the family is empty. Thus there are at most $1 + \alpha^{-1}\log W_0$ successful insertions, up to rounding, and at most one failed call. This formulation also covers $W_0 = 1$.

Using Equation (15), the number of calls is

$$O\left(\frac{n}{\sqrt{k}}(n/D + k)\log(2n)\right).$$

The threshold is at least $\sqrt{k}/(8n) \geq 1/(8n)$ and below one, so each call costs $O(\log(2n))$ original queries. Multiplication gives Equation (28). □

When $k = 1$ we have $k' = 0$, $b = f - 1$, and every witness has empty padding: Equation (12) and $c_H(S) \leq b$ force $c_H(S) = b$. The final phase therefore either reaches flow value f or refutes it. There is no separate randomized finishing step.

8 Finding the value, a flow, and a shore

The next lemma runs the phases with the halving schedule of [14, Corollary 5.2].

Lemma 17 (Threshold decision)**.** *After preprocessing, a target $1 \le f \le n-1$ can be decided using*

$$O\Big((n^2/D + n\sqrt{f})\log^2(2n)\Big) \tag{30}$$

queries. A positive answer leaves an explicit learned graph with flow value at least f. All edges learned by earlier decisions may be retained.

Proof. Start with $k = f$. The invariant $\nu(H) \ge f - k = 0$ holds for any learned graph. Apply Theorem 16. A failure gives a correct negative answer. After success replace k by $\lfloor k/2 \rfloor$. The phase guarantee is exactly the invariant for the new k. When $k = 0$ the explicit graph has $\nu(H) \ge f$, which proves a positive answer because $H \subseteq G$.

If the successive positive deficits are $k_0, k_1, \ldots, k_m$, then $k_0 = f$, $k_{i+1} = \lfloor k_i/2 \rfloor$, and

$$\sum_i \sqrt{k_i} = O(\sqrt{f}), \qquad \sum_i \frac{1}{\sqrt{k_i}} = O(1). \tag{31}$$

For the first estimate, use $k_i \le f/2^i$. For the second, read the sequence backwards and use $k_i \ge 2k_{i+1}$; the final positive term is one. Summing Equation (28) proves Equation (30). Retaining learned edges preserves Equation (5) and the same family $\mathcal{A}$. At the start of the next decision $k = f$ again, so its initial invariant remains valid. □

8.1 Recovering a minimum shore from an explicit maximum flow

It is not enough to minimize a cut in the learned graph: an unknown edge could increase its value in G. We instead discover the set reachable in the true residual graph of the explicit maximum flow, by the residual search of Anand, Saranurak, and Wang [4, Corollary 4.3]; by the max-flow min-cut theorem [9], this set is a minimum shore.

Lemma 18 (Residual reachability)**.** *Given an explicit integral maximum s–t flow g in G, a minimum terminal shore can be recovered with $O(n\log(2n))$ cut queries.*

Proof. Let F be the undirected unit graph on the edges with $|g_{uv}| = 1$. It is explicit. For an edge used from u to v, the residual network has only the reverse direction v to u, of residual capacity two. Every edge not in F is unused and has residual capacity one in both directions. Thus, for reachability, the residual graph consists of known reverse arcs together with the undirected graph $U = G \setminus F$. Its cuts satisfy $c_U(S) = c_G(S) - c_F(S)$.

Initialize $A = \{s\}$ and close A under all known reverse arcs, using no queries. Test whether $c_U(A)$ is zero. If it is positive, use Lemma 3 to find an edge of U between A and its complement, add its outside endpoint to A, close again under known arcs, and repeat. Each successful edge search adds a new vertex; there are at most $n-1$ of them, and one final zero test. This proves the query bound.

Every added vertex is reachable from s. On termination there is neither a reverse arc nor an unused undirected edge leaving A, so A is exactly the reachable set in the residual network. Since g is maximum, $t \notin A$. Every original edge crossing from A to its complement must carry one unit of flow in that direction: an unused edge or an edge carrying reverse flow would supply a residual arc leaving A. Summing divergence over A gives $c_G(A) = \mathrm{val}(g) = \nu(G)$. Return A, or its complement if a t-oriented shore is desired. □

Proof of Theorem 1. Set

$$D = \max\{2, \lceil \sqrt{n} \rceil\}.$$

For every $n \geq 2$ this lies in $[2, n]$. Perform Lemma 5. If the core is empty, the entire graph is explicit, and an exact minimum cut and integral maximum flow can be computed offline. Otherwise retain K, J, C and the family $\mathcal{A}$.

The initial degree queries give

$$\nu(G) \leq U := \min\{d_G(s), d_G(t)\} \leq n - 1. \tag{32}$$

Binary search the integer optimum with feasible lower endpoint zero and infeasible upper endpoint $U + 1$. Every tested target is in $[1, U]$, so Lemma 17 applies. There are $O(\log(2n))$ decisions. Retain all learned edges, including those learned in a decision that eventually refutes its target.

On termination the lower endpoint is $\nu(G)$. If it is positive, a previous successful decision supplied an explicit subgraph supporting that value; its edges are still present. If it is zero, the zero flow is already explicit. Because the final learned graph is a subgraph of G, an offline integral maximum-flow computation in it yields a maximum flow of exactly $\nu(G)$ in G. Apply Lemma 18 to obtain a minimizing shore in the original graph.

Taking the worst case $f \leq n - 1$ in Equation (30), the total query count is

$$\begin{aligned} &O\Big(nD \log(2n) + (n^2/D + n^{3/2}) \log^3(2n) + n \log(2n)\Big) \\ &\quad = O\Big(n^{3/2} \log^3(2n)\Big). \end{aligned}$$

All stages are deterministic and use exact queries and arithmetic. □

For $n = 2$, $D = 2$ forces the core to be empty; the single possible edge is determined by a degree query. More generally the empty-core branch handles every instance fully exposed by peeling. If $U = 0$, no target decision is needed. The zero flow followed by residual reachability still returns a cut of value zero, even when a nonempty core exists. The same argument covers disconnected graphs and isolated terminals.

9 From terminal flows to all pairs

The canonical witness bound concerns a simple unit graph and a target of at most $n - 1$. It cannot be applied unchanged after replacing each terminal by an arbitrarily capacitated star. The bridge to all pairs is a separate extension of the residual-flow analysis: terminal capacities are known, and only internal edges require discovery.

Theorem 19 (Exact flow with known heavy terminal stars)**.** *Let J be a simple unknown graph on $q \geq 1$ internal vertices, give each internal edge a known integer capacity $C \geq 1$, and add known nonnegative integral capacities a_v from a new source and z_v to a new sink. If $F = \sum_v a_v$, an explicit integral maximum flow and a minimum source shore can be found deterministically using*

$$O\Big((q + F + C^{1/3} q^{5/3} + q\sqrt{CF}) \log(2q + 2)\Big)$$

queries to J. More particularly, if $q \leq t^3$, $1 \leq C \leq R$, $F \leq qt$, and $t, R \geq 1$ are integers, the cost is $O(qRt^2 \log(2q + 2))$.

The proof in Section 12 explicitly implements residual searches and blocking phases, extending the cut-query blocking-flow analysis of [4] to heavy known terminal stars. Large capacities on the two stars do not enter the internal layer-pair bound. This is necessary for the following construction, whose matching demand is $qb/2$ rather than $O(q)$.

We run the cut-matching game of [22] with the KKOV cut player [21], as in [4, Appendix A], whose perfect b-matchings our clones represent. For a part W, create b labeled clones of each vertex. A bisection of these clones determines a_v source clones and $z_v = b - a_v$ sink clones. An exact heavy-star flow either exposes a sparse, sufficiently balanced split of W, or routes almost a perfect matching across the bisection. Complete the latter with recorded fake pairs. The cut player receives a legal perfect matching in every completed round. Its output expands the clone set, and its real paths have at most R congestion. Projecting only the fake pairs yields an explicit multigraph D such that, simultaneously for every $T \subseteq W$,

$$b \min\{|T|, |W \setminus T|\} \leq R c_{G[W]}(T) + c_D(T). \tag{33}$$

The multigraph, its degree table, and its cardinality are constructed from the executed rounds; none is supplied as an oracle certificate.

A second heavy-star flow uses capacity R on original edges, source capacity $d_D(v)$, and sink capacity $b/2$. It takes the place of the expander pruning [30] used in [4], and is solved by one exact flow, as in the trimming step of Saranurak and Wang [30]. A minimum source shore removes a set P of size at most $|W|/8$ and boundary at most $b|P|$. Its complement is an expander in its *induced* graph. Finish that complement and recurse on P. Failed games recurse on both sides of their sparse cut, and their already executed flows remain charged. The disjoint active parts and their uniform shrink establish the level-mass bound; a separate smaller-side charging argument bounds interleaf edges. This recursion and charging follow the expander decomposition of [30].

As in the friendly-cut sparsifiers of [2], we then shave vertices of small degree or excessive boundary degree. No resulting block splits a friendly cut of value at most $2N$, where friendly means, as in [20], that every vertex retains at least one sixth of its neighbors on its own side. All edges between the final blocks are learned with their original endpoints. For each block, replace its hidden internal edges by a known star whose incident capacities are its internal degrees; this is the star transform of [20]. This star graph dominates every original cut and admits an exact extension of a minimum cut for each original terminal pair. Hence every minimum shore found offline in the star graph restricts to a true original minimum shore, with equality of cut values. The full proof, including padding, integer parameters, and the query cap, concludes in Section 15.

10 Scope, dependencies, and limitations

The unit and simplicity assumptions enter different parts of the proof. Simplicity relates a part's number of vertices to its internal degrees in Lemma 7. Unit capacities give $\nu(G) \leq n-1$, the padding universe of ordinary vertex pairs, integer phase thresholds, and an integral residual augmentation with positive weights at least one. They also turn a value-f flow into f edge-disjoint paths. The proof therefore does not give a weight-independent extension to arbitrary weighted graphs, nor an extension to directed cut oracles.

The exact oracle uses only $O(\log(2n))$ answer bits. Witness multiplicities and marginal potentials are computed as finite integers and rationals, and the long-edge bins and comparisons can be formed exactly. Nevertheless, the canonical family and its witnesses can be exponentially large. The theorem is a bound on adaptive query complexity and makes no polynomial-time claim.

The single-pair contribution is the minimum-degree preprocessing, fixed canonical extensions, and resulting restricted witness count, combined with the JNS method. The external mathematical inputs are integral max-flow/min-cut and residual augmentation [9], the Tutte–Nash-Williams tree-packing theorem [31, 27], the JNS witness framework, and the JNS long-edge construction [14]. The support estimate and the version-specific long-edge construction are reproduced here; the classical general max-flow and tree-packing theorems are cited rather than reproved. Related global-cut and certificate results serve as context and are not used to establish Theorem 1.

This manuscript is a mathematical research draft. Exact finite checks of the source argument exercise canonicalization, feasible and infeasible targets, padding, and witness loss. Such checks do not prove the asymptotic query bound, and their exhaustive edge search is not an implementation of the query-efficient long-edge routine. Appendix A records the correspondence with the Lean library, including both main theorems and every numbered intermediate statement. It distinguishes direct counterparts from differences of formulation and records the checks performed for this revision. The randomized Gomory–Hu reduction [28] supplies no dependency to Theorem 2.

11 The deterministic long-edge construction

We give the construction behind JNS Claim 5.7 and Lemma 5.8, using Lemma 3 for its only graph access. This section also makes explicit why probabilistic counting is compatible with a deterministic algorithm in the present computation model.

Lemma 20 (Separating assignments; JNS Claim 5.7)**.** *For each integer $\ell \geq 2$, there is a family of $O(\log(2\ell))$ assignments*

$$A = ((p_0, q_0), \ldots, (p_{\ell-1}, q_{\ell-1})), \qquad (p_i, q_i) \in \{1,2,3\} \times \{1,2\},$$

with $p_i \neq p_{i+1}$, such that for every i, j with $|i-j| \geq 2$ some assignment has $p_i = p_j$ and $q_i \neq q_j$. The family can be fixed deterministically without graph queries.

Proof. Set $p_0 = 1$ and choose each subsequent p_i uniformly from the two colors other than p_{i-1}. Choose the q_i independently and uniformly from $\{1,2\}$. For fixed $j < i$ with $i - j \geq 2$, the Markov construction gives

$$\Pr(p_i = p_j) = \tfrac{1}{2}\Pr(p_{i-1} \neq p_j) = \tfrac{1}{4} + \tfrac{1}{4}\Pr(p_{i-2} = p_j) \geq \tfrac{1}{4}.$$

Independence of the q labels implies that this pair is separated in the required way with probability at least $1/8$. Choose

$$t = 32\lceil \log_2(2\ell) \rceil$$

independent assignments. The probability that any of the fewer than ℓ^2 eligible pairs is missed is at most $\ell^2(7/8)^t \leq \ell^2 e^{-t/8} < 1$. A family of this size therefore exists. Enumerate the finitely many families of size t in a fixed order and take the first satisfying the condition. Checking the condition depends only on ℓ, so this is a deterministic, query-free computation. □

Proof of Lemma 15. Put $\eta = \delta/2$ and assign a vertex v to bin

$$j(v) = \lfloor \phi_v/\eta \rfloor. \tag{34}$$

The indices lie in $\{0, \ldots, \ell-1\}$ for $\ell = \lfloor 1/\eta \rfloor + 1 = O(1/\delta)$. This floor convention includes $\phi_v = 1$; when $1/\eta$ is an integer, the final bin is the singleton $\{1\}$. Two elementary implications are

$$|\phi_u - \phi_v| \geq \delta \implies |j(u) - j(v)| \geq 2, \tag{35}$$

$$|j(u) - j(v)| \geq 2 \implies |\phi_u - \phi_v| \geq \delta/2. \tag{36}$$

For example, if $j(u) \geq j(v) + 2$, then $\phi_u \geq (j(v) + 2)\eta$ and $\phi_v < (j(v) + 1)\eta$; the endpoint-one bin causes no exception to these inequalities.

Fix a family of separating assignments from Lemma 20. For each assignment A and $r \in \{1, 2, 3\}$, form the disjoint sets

$$V^A_{r,1} = \{v : (p_{j(v)}, q_{j(v)}) = (r, 1)\}, \qquad V^A_{r,2} = \{v : (p_{j(v)}, q_{j(v)}) = (r, 2)\}.$$

Test whether R contains an edge between these two sets. A zero test costs at most three cut queries by Equation (3). At the first positive test, find one such edge with Lemma 3, return it, and stop. If all tests are zero, return FAIL.

For a returned edge, the bins have the same p label but distinct q labels. They are distinct, and cannot be adjacent because adjacent bins have different p labels. Its length is therefore at least $\delta/2$ by Equation (36). Conversely, an edge of length at least δ has bin indices differing by at least two. Some assignment matches those indices, so its endpoints lie in one tested pair of sets and that test is positive. Thus failure implies that all edge lengths are strictly below δ.

There are $O(\log(2\ell)) = O(\log(2/\delta))$ tests. Every negative test costs $O(1)$ queries, and there is at most one positive test, after which the $O(\log(2n))$ edge search terminates the routine. The total is $O(\log(2/\delta) + \log(2n)) = O(\log(n/\delta))$. All bin calculations are exact rational operations. All assignment selection occurs offline, before querying R. □

The distinction between a zero test and an edge search matters: charging $O(\log n)$ to every empty test would introduce an unnecessary product of logarithms. The test also uses the ordinary cut oracle of an undirected graph. In the main algorithm that graph is always $G \setminus H$; directed residual networks enter only the existence proof of Lemma 13, not this oracle interface.

12 Exact flows with heavy known terminal stars

This section proves Theorem 19. The two added terminals are denoted σ, τ to distinguish them from an eventual input pair. Edges σv and $v\tau$ have known capacities a_v, z_v; an internal edge has capacity C. We use signed undirected flows. Equivalently, replace each edge by two oppositely directed arcs of its capacity and cancel opposite flow. Either convention has internal residual capacities at most $2C$. Integral max-flow/min-cut and cycle cancellation are the classical facts used here [9].

12.1 The actual residual-query implementation

For a shore S of the augmented network, its capacity is C times the cut of $S \cap W$ in J, plus known terminal contributions. For disjoint sets A, B of network vertices, the total residual capacity from A to B under an explicit signed flow g is

$$r_g(A, B) = \tfrac{1}{2}(c(A) + c(B) - c(A \cup B)) - \sum_{u \in A, v \in B} g_{uv}. \tag{37}$$

All entries are nonnegative. Thus one can test a residual rectangle in three input queries and locate a positive entry by binary search, as in [4, Lemmas 3.3 and 4.1]. A located internal edge has its existence determined by the query transcript; updating its flow never requires access to an unseen edge. Known star entries require no input query.

Breadth-first search from σ, as in [4, Corollary 4.3], maintains the undiscovered vertices. For each discovered vertex, repeatedly find a positive residual neighbor in that set, remove the neighbor from the set, and enqueue it. Each successful search discovers a new vertex, and each processed

row has at most one final empty test. There are $O(q)$ such events, so a BFS costs $O(q \log(2q+2))$ queries. This argument includes the BFS that finally certifies that τ is unreachable.

The phases are Dinic's blocking phases [7], searched as in [4, Claim 4.9]. In a finite-distance phase, retain the BFS levels and perform depth-first search using only arcs to the next level. At a dead end mark the vertex dead for this phase and backtrack; search a row only among live vertices of the next level. On reaching τ, augment one integral unit along the discovered path and restart the path search. Augmenting one unit is sufficient even when capacities are large. Each augmentation removes at least one unit of demand, and its length is the current distance d. Reverse residual arcs created by augmentation go to the previous level, so they cannot revive a dead vertex or a deleted admissible arc. Each dead-end event is therefore charged once per vertex; every other search and update is charged to a path arc. A phase augmenting a units costs

$$O((q + da) \log(2q+2)). \tag{38}$$

It terminates when no layered source–sink path remains, and hence is a blocking phase. Positive residual arcs after the phase advance the old level by at most one. A path of length d would advance by one on every step and would therefore be a surviving layered path, a contradiction. Consequently the next finite distance strictly increases. These are cut-query versions of the blocking-flow operations in ASW [4]; Equations (37) and (38) account for all their graph access.

12.2 Internal layer cuts and augmentation tails

The next lemma is the layer argument of Karzanov [17] and Even and Tarjan [8], in the form of [4, Lemma 4.8]; the point is that the heavy terminal arcs do not enter it.

Lemma 21 (Heavy-star residual bound)**.** *If the residual distance is $d \geq 4$, there is a source shore of residual capacity at most $8Cq^2/d^2$. In particular the remaining flow deficit is at most this number.*

Proof. Let $L_0 = \{\sigma\}, L_1, \ldots, L_d$ be the reachable BFS levels, with $\tau \in L_d$. Among the $m = \lfloor (d-1)/2 \rfloor$ disjoint pairs $(L_1, L_2), (L_3, L_4), \ldots, (L_{2m-1}, L_{2m})$, one has total size at most q/m. Both levels in every such pair consist of internal vertices. The outgoing residual arcs of the prefix through L_{2i-1} all lead to L_{2i}. An arc to a later level would contradict the BFS distances; an arc to an unreachable vertex would make that vertex reachable. No heavy terminal arc is a positive outgoing arc of this prefix.

Simplicity gives at most $|L_{2i-1}||L_{2i}|$ internal pairs, each of residual capacity at most $2C$. The selected prefix thus has residual capacity at most

$$2C|L_{2i-1}||L_{2i}| \leq \frac{Cq^2}{2m^2} \leq \frac{8Cq^2}{d^2},$$

since $m \geq d/4$ for $d \geq 4$. Every additional feasible flow is bounded by every residual source cut, proving the deficit bound. □

Write (d_i, a_i) for the actual positive blocking phases. Distances increase strictly and $\sum_i a_i \leq F$. For every integer $k \geq 3$, apply Lemma 21 just before the first phase with $d_i > k$. Every augmentation in that phase and all later phases is bounded by the then remaining optimum deficit. If there is no such phase the sum is zero. Therefore

$$\sum_{i:d_i>k} a_i \leq \min\{F, 8Cq^2/(k+1)^2\}. \tag{39}$$

For every integer $K \geq 4$, summing tails gives

$$\sum_i d_i a_i \leq KF + \sum_{k=K}^{\infty} \frac{8Cq^2}{(k+1)^2} \leq KF + \frac{8Cq^2}{K}. \tag{40}$$

This is an inequality about the executed phases, not a choice of a favorable maximum flow from the hidden graph.

As in the classical phase bound [17, 8], let $h \geq 4$ be an integer. There are at most h positive phases of distance at most h. At the first later phase, the deficit is at most $8Cq^2/h^2$, and each subsequent positive phase augments at least one integer unit. Thus

$$\#\text{positive phases} \leq h + 8Cq^2/h^2. \tag{41}$$

Take $h = \max\{4, \lceil (8Cq^2)^{1/3} \rceil\}$ to get $O(1 + C^{1/3}q^{2/3})$ phases. For $F > 0$, choose

$$K = \max\{4, \lceil q\sqrt{8C/F} \rceil\}.$$

Then Equation (40) gives $O(F + q\sqrt{CF})$. When $F = 0$ the zero flow and source-only shore solve the problem directly. Combining the phase count, path work, and final BFS proves the first bound in Theorem 19. At termination the residual reachable shore has no positive outgoing residual arc, so its capacity equals the value of the explicit flow; it is an exact minimum cut.

For the part-size-linear form, use the global integer thresholds

$$h = 8Rt^2, \qquad K = 4Rt. \tag{42}$$

They are used only in the analysis; the flow algorithm runs to completion. Since $q \leq t^3$, $C \leq R$, and $F \leq qt$,

$$\frac{8Cq^2}{h^2} \leq t^2, \qquad KF + \frac{8Cq^2}{K} \leq 4Rqt^2 + 2q^2/t \leq 6Rqt^2.$$

There are at most $9Rt^2$ positive phases, and the sum of path lengths is at most $6Rqt^2$. The final BFS and Equation (38) give $O(qRt^2 \log(2q+2))$, proving the second assertion. The network has $q+2$ vertices, regardless of how many terminal clones will later be represented by its known stars.

13 The deterministic cut player and its legal responses

The cut-matching game is that of Khandekar, Rao, and Vazirani [22]. We give the entropy argument of KKOV [21, Section 4] and its normalization explicitly, because constant repetition is part of the algorithm and its query accounting. One full run below (four balanced subgames, removal of sparse sets, and a final bisection) is the strategy of [21, Theorem 4.1 and Figures 3–4]; the four fresh full runs are our normalization. A matching is a fixed-point-free involution on an even set of $m \geq 2$ labeled vertices. A legal response to a bisection pairs every vertex with a vertex of the opposite side. The history graph is the *multigraph sum* of the returned perfect matchings.

Lemma 22 (Normalized deterministic cut player)**.** *There is a deterministic strategy making $O(\log(2m))$ bisection requests such that every sequence of legal responses produces a multigraph X with*

$$c_X(S) \geq \min\{|S|, m - |S|\} \quad (S \subseteq [m]).$$

The strategy examines only its explicit matching history. It consists of four fresh full runs; each full run consists of four fresh balanced subgames, offline pruning, and one final matching request.

Proof. For a matching M, average a probability vector p across its pairs: $p'(v) = (p(v) + p(Mv))/2$. Begin with a point mass at each starting vertex u, and perform these lazy matching steps along the history. Write p_u for the resulting row and $\Phi = \sum_u H(p_u)$, where $H(p) = -\sum_v p(v) \log p(v)$ and $0 \log 0 = 0$. The transition matrices are doubly stochastic, so every row sums to one, every column sums to one, and $0 \le \Phi \le m \log m$. Concavity of $-x \log x$ implies that Φ never decreases.

For a fixed shore S, put $\mu(v) = \sum_{u \in S} p_u(v)$. Then $0 \le \mu(v) \le 1$. One lazy matching step increases its mass outside S by at most half the number of matching edges crossing S. Summing over the history H yields the leakage inequality

$$\sum_{u \in S} p_u(S^c) \le \tfrac{1}{2} c_H(S). \tag{43}$$

Suppose $m/4 \le |S| \le m/2$ and $c_H(S) < |S|/4$. At least $|S|/2 \ge m/8$ of its starting rows have $p_u(S^c) \le 1/4$. Complete S to a bisection and request a matching across it. Every vertex of S is then matched outside S.

Fix one such row p. The mass in S is at least $3/4$, whereas $\sum_{v \in S} p(Mv) \le 1/4$. The vertices of S with $p(v) < 2p(Mv)$ have total mass at most $1/2$, leaving at least $1/4$ mass on vertices with $p(v) \ge 2p(Mv)$. If

$$g(x, y) = 2\big(-(x + y)/2 \cdot \log((x + y)/2)\big) + x \log x + y \log y,$$

then $g \ge 0$ by concavity, $g(cx, cy) = cg(x, y)$, and $g(1, r) > 0$ for $0 \le r \le 1/2$ by strict concavity. Continuity on this compact interval supplies a universal $\gamma > 0$ with $g(x, y) \ge \gamma x$ whenever $x \ge 2y \ge 0$. The sum of $g(p(v), p(Mv))$ over all vertices is twice the entropy gain. It is at least $\gamma/4$, so this row gains at least $\gamma/8$. All other rows gain a nonnegative amount. A requested round therefore increases Φ by at least $\gamma m/64$.

A balanced subgame starts with empty history, searches exhaustively for the displayed sparse quarter-balanced shore, and stops when none exists. It makes at most $T(m) = \lceil 64 \log m/\gamma \rceil + 1$ requests, since otherwise the potential would exceed $m \log m$. All selections are deterministic finite searches on explicit graphs. The analysis uses entropy; the algorithm only compares integer cut sizes and cardinalities.

Run four such balanced subgames with fresh empty local histories. Their union H satisfies $c_H(S) \ge |S|$ whenever $m/4 \le |S| \le m/2$. Starting with $P = \varnothing$, repeatedly remove a nonempty $S \subseteq [m] \setminus P$ with $|S| \le (m - |P|)/2$ and $c_{H[[m] \setminus P]}(S) < |S|/2$. The invariant is $c_H(P) \le |P|/2$ and $|P| < m/4$. Indeed, for $Q = P \cup S$,

$$c_H(Q) = c_H(P) + c_{H[[m] \setminus P]}(S) - e_H(S, P) \le |Q|/2.$$

If $m/4 \le |Q| \le m/2$, quarter-density contradicts this inequality. If $|Q| > m/2$, then $|Q| \le (m + |P|)/2 < 5m/8$, so $3m/8 < |Q^c| < m/2$ and quarter-density gives $c_H(Q) = c_H(Q^c) \ge |Q^c| > 3m/8 > |Q|/2$, another contradiction. Each removal reduces the remaining cardinality, so this process stops. Its remaining set U has induced edge expansion at least $1/2$.

Complete P to a bisection and request one final matching M. Every vertex of P is paired into U. For any T with $k = |T| \le m/2$, let $r = |T \cap U|$. At most r vertices in $T \cap P$ can be matched within T, whence $c_M(T) \ge k - 2r$. Also $c_H(T) \ge \frac{1}{2} \min\{r, |U| - r\}$. If $r \le |U|/2$ and $r \ge k/2$, the latter bound is at least $k/4$. If $r \le |U|/2$ and $r < k/2$, the two bounds add to at least $k - 3r/2 > k/4$. Finally, if $r > |U|/2$, then $|U| - r \ge 3m/4 - k \ge k/2$, and again $c_H(T) \ge k/4$. Thus one full run produces expansion $1/4$ in at most $4T(m) + 1$ rounds.

Execute four fresh full runs and add their matching multigraphs. Their expansions add to one and the number of actual requests is at most $16T(m) + 4$. Fresh means that a subgame's potential

starts at its own empty history; its requests may still carry the entire previous history, and the legal response may depend on it. This proves the claim against fully adaptive legal matching responses. Copying one completed run four times would not be this execution and is not used. □

Fix a public round bound $R \geq 2$ valid for every even clone count at most Nb, with $R = O(\log(2Nb))$. It may be chosen as an integer constant times a ceiling base-two logarithm, using a fixed integer upper bound for $64/\gamma$. The total number of rounds below always includes all sixteen balanced subgames and all four final requests.

The next lemma is the matching player of [4, Theorem A.4]: one exact flow either exposes a sparse balanced cut or routes an almost perfect b-matching, which fake pairs complete.

Lemma 23 (Executed matching round)**.** *Let $J = G[W]$, $q = |W|$, and let b be a positive even integer. For a bisection (A, B) of its qb clones, put $a_v = |A \cap \mathrm{clones}(v)|$ and $z_v = b - a_v$. Given an integer $\beta \geq 0$, an exact unit-internal-capacity heavy-star flow returns either*

(i) a shore $S \subset W$ with both sides larger than β and $c_J(S) < b \min\{|S|, q - |S|\}$; or

(ii) a legal perfect matching across (A, B), with at most βb fake pairs and an explicit routing of every real pair with congestion at most one on the original edges in this round.

Proof. Both terminal stars have total capacity $F = qb/2$. Let ν be the exact maximum-flow value. If $\nu < F - \beta b$, an exact minimum source shore $\{\sigma\} \cup S$ gives, with $\Delta = F - \nu > \beta b$,

$$\nu = F - a(S) + z(S) + c_J(S), \qquad c_J(S) = a(S) - z(S) - \Delta. \tag{44}$$

The imbalance is at most $b|S|$, and, since total imbalance is zero, also at most $b(q-|S|)$. Nonnegativity of the cut and $\Delta > \beta b$ force both sides to exceed β, and prove the strict boundary bound. This is the split branch; no matching is returned to the cut player.

Otherwise cancel cycles in the explicit integral flow and decompose it into ν unit paths. Assign distinct source clones to their first internal vertices and distinct sink clones to their last internal vertices. The star capacities guarantee enough of each kind. Pair these endpoints. No internal edge is used by two paths because its capacity is one, after opposite flow is cancelled. A path σ, v, τ uses no internal edge and pairs distinct clones of the same vertex; it is allowed. Exactly $F - \nu$ clones on each side remain unmatched. Pair them in the fixed label order and mark precisely these pairs fake. This is a genuine perfect matching across the requested bisection, with at most βb fake pairs. The paths and marker set are part of the public transcript. □

As in [4, proof of Lemma A.1], the fake pairs are few and the real pairs of all rounds embed in J with bounded congestion.

Lemma 24 (Constructed fake multigraph)**.** *Suppose the normalized game completes on W in $r \leq R$ rounds using Lemma 23(ii). Project every marked fake pair to its original endpoints, retaining multiplicity and discarding projected loops; call the result D, and put $M = |E(D)|$. Then*

$$M \leq R\beta b, \qquad d_D(v) \leq Rb, \qquad Rc_J(T) + c_D(T) \geq b \min\{|T|, q - |T|\} \quad (T \subseteq W). \tag{45}$$

Proof. Count every fake pair once, from its A endpoint in its own round. The count is exactly that round's deficit $F - \nu$, before loops are discarded, so the total is at most $R\beta b$. A vertex has b clones, each incident to one matching edge per round. Therefore its projected fake degree is at most Rb. In particular $\sum_v d_D(v) = 2M$; a projected loop contributes neither a graph edge nor a degree.

For an original shore T, lift it to all clones of its vertices. Its smaller clone side has size $b \min\{|T|, q - |T|\}$. By Lemma 22, at least this many matching edges cross the lift. Each real

crossing pair has a path crossing $\delta_J(T)$, and distinct paths within a round can be charged to distinct crossed original edges. There are at most R rounds, so these pairs number at most $Rc_J(T)$. A fake pair crosses the clone lift exactly when its projected nonloop edge crosses T, with multiplicity. Its contribution is precisely $c_D(T)$. This proves the last inequality for all shores simultaneously. □

14 Pruning, the partition, and its complete query budget

The next flow takes the place of the expander pruning of Saranurak and Wang [30] used in [4]. As in the trimming step of [30], a single exact maximum flow and its minimum cut certify an induced expander; here the fake edges play the role of the boundary edges in [30].

Lemma 25 (One-flow fake-edge pruning)**.** *Assume the last inequality of Equation (45) and $d_D(v) \le Rb$. Give each edge of J capacity R, add source capacities $a_v = d_D(v)$ and sink capacities $b/2$, and find an exact maximum flow and minimum source shore $\{\sigma\} \cup P$. For $U = W \setminus P$,*

$$|P| \le 4M/b, \qquad c_J(P) \le b|P|, \tag{46}$$

$$c_{J[U]}(T) \ge \frac{b}{4R} \min\{|T|, |U \setminus T|\} \qquad (T \subseteq U). \tag{47}$$

If $P \neq \varnothing$, the boundary inequality is strict.

Proof. Put $A = \sum_v a_v = 2M$. The source-only cut has capacity A. Optimality of the returned shore gives

$$\tfrac{b}{2}|P| + Rc_J(P) \le a(P) \le A. \tag{48}$$

It follows that $|P| \le 4M/b$. The other degree bound gives $c_J(P) \le (b - b/(2R))|P|$, proving the boundary assertion.

Every edge leaving a minimum source shore is saturated in the forward residual direction. Thus the source edge into each $v \in U$ carries a_v, and every original edge from P to U carries R toward U. Sum conservation over $T \subseteq U$. Its sink flow is at most $(b/2)|T|$, and its net flow toward $U \setminus T$ is at most the capacity of that internal cut. Consequently

$$a(T) + Re_J(T, P) \le \tfrac{b}{2}|T| + Rc_{J[U]}(T). \tag{49}$$

For $|T| \le |U|/2$, also $|T| \le |W|/2$. The fake-edge premise and $c_D(T) \le a(T)$ give

$$b|T| \le Rc_{J[U]}(T) + Re_J(T, P) + c_D(T) \le 2Rc_{J[U]}(T) + \tfrac{b}{2}|T|.$$

Rearrange, and apply the same inequality to $U \setminus T$ for larger T. The conclusion is expansion of the induced graph $J[U]$, which is stronger than a statement about cuts of J with outside edges included. □

14.1 The public recursive construction

Work for now on a simple graph G of $N \ge 2$ vertices and minimum degree at least two. Let t be the least positive integer with $N \le t^3$, put $b = 2t$, and choose R as above for all clone counts at most Nb. These parameters precede every oracle reply. The recursion follows the expander decomposition of [30]: a balanced sparse cut recurses on both sides, and after trimming only the small side recurses. For a part W of size q, do the following, with all order choices fixed. If $q \le 64R$, finish W as a small leaf without running a game. Otherwise put

$$\beta = \left\lfloor \frac{q}{32R} \right\rfloor \tag{50}$$

and execute the normalized cut player and its actual matching flows. On the first split response of Lemma 23, abandon this game and recursively process both sides of the split. If all rounds complete, construct D from their marker sets and run Lemma 25. Finish U as a good leaf; recursively process P if it is nonempty.

In the completed branch, $M \le R\beta b \le qb/32$, so $|P| \le q/8$ and U is nonempty. The pruning source capacity is $2M \le qb/16 \le qt$. Every matching call also has source capacity $qb/2 = qt$. Thus all calls meet Theorem 19 with $q \le t^3$ and internal capacity either 1 or R. No balance assumption is imposed on the pruning stars.

At every stage active parts and finished leaves are disjoint and cover $V(G)$. A sparse split replaces its part by two disjoint nonempty children. A pruning split replaces its part by the nonempty finished set U and the optional active child P. Induction therefore gives a genuine partition at termination. Choosing one member as the representative of each nonempty leaf gives an idempotent map onto the representatives; there is no assumed or externally selected favorable partition.

Lemma 26 (Depth, mass, and boundary)**.** *The active recursion has depth $O(R\log(2N))$, and its part sizes sum to at most $O(NR\log(2N))$. If B_0 is the number of edges joining distinct final leaves, then*

$$B_0 \le bN\lceil \log_2 N\rceil. \tag{51}$$

All matching and pruning work uses $O(Nt^2R^3\log^2(2N))$ input queries.

Proof. For $q > 64R$, Equation (50) implies $q/(64R) \le \beta \le q/(32R)$. In a failed game, both children have size greater than β, so each has size at most $(1-1/(64R))q$. The pruning child has size at most $q/8$, which satisfies the same shrink inequality. Thus every active path has $O(R\log(2N))$ nodes. At a fixed depth the parts are disjoint subsets of the original vertex set; their cardinalities sum to at most N. Summing over depths proves the mass bound, including parts whose games later fail.

Every sparse split cuts fewer than b times the smaller child size in edges. A pruning split has smaller side P and cuts at most $b|P|$ edges. As in [30], charge the crossing edges at either split to vertices of its smaller child, at most b per charged vertex. Whenever a vertex is charged, its containing part at least halves. If that child is finished, it is never charged again. An original edge between different final leaves is counted at exactly its first separating split. Each vertex can be charged at most $\lceil \log_2 N\rceil$ times, proving Equation (51). This argument does not assume logarithmic recursion depth.

A node makes at most R matching-flow calls, including the call that returns a splitting cut, and at most one pruning-flow call. Each costs $O(qRt^2\log(2N))$ by Theorem 19. Hence the total is

$$O\left((R+1)Rt^2\log(2N)\sum_{\text{active nodes}} q\right) = O(Nt^2R^3\log^2(2N)).$$

If a game is abandoned, every earlier call at that node has been included; there is no refund for unsuccessful rounds.

Finally, a local induced cut is always simulated directly by

$$c_{G[W]}(T) = \tfrac{1}{2}\big(c_G(T) + c_G(W\setminus T) - c_G(W)\big), \qquad T \subseteq W. \tag{52}$$

This costs three original queries independently of the depth. Scaling by R and adding known stars are offline arithmetic. The residual rectangle routine uses only a constant number of these local cuts; there is no recursively multiplied simulation overhead. □

15 Shaving, original shores, and the all-pairs theorem

A shore S is *friendly* if every vertex has at least $d_G(v)/6$ neighbors on its own side. Equivalently its crossing degree $\mathrm{cr}_S(v)$ is at most $5d_G(v)/6$. Empty and full shores are allowed in this definition. This is the 1/6-friendliness of [20]; friendly cuts and friendly-cut sparsifiers were introduced by Abboud, Krauthgamer, and Trabelsi [2]. A partition respects a shore if the shore is a union of its blocks.

15.1 A simultaneously valid friendly partition

Set

$$w = 2N, \qquad \theta = 32Rt^2. \tag{53}$$

Every leaf W has the following small-cut-side property:

$$c_{G[W]}(T) \le 2N \quad \Longrightarrow \quad \min\{|T|, |W \setminus T|\} \le \theta. \tag{54}$$

For a small leaf the smaller side is at most $32R \le \theta$. For a good leaf, Equation (47) bounds it by $8RN/b = 4RN/t \le 4Rt^2 \le \theta$. This uniform parameter also covers all leaves when the entire input is smaller than $64R$.

The shaving rule and Lemma 27 follow the deterministic friendly-cut sparsifier of [2]: shave vertices of low degree or with a large fraction of their degree leaving the expander, and contract the rest. The constants $60 = 10 \cdot 6$ and $24 = 4 \cdot 6$ are those of [20, Algorithm 1] for friendliness 1/6. Kenneth-Mordoch and Krauthgamer [19, Lemma 1.7] construct such sparsifiers with randomized cut queries.

Query the degree $d(v)$ and degree $d_{\mathrm{out}}(v)$ to the complement of v's leaf. These take $O(N)$ cut queries using the rectangle identity. In every leaf retain exactly the vertices with

$$d(v) \ge 60\theta, \qquad d_{\mathrm{out}}(v) \le d(v)/24. \tag{55}$$

Make the retained vertices of a leaf one block if the set is nonempty; every other vertex is a singleton. In a small leaf none can be retained: retention would imply at least $23d(v)/24 \ge (23/24)60\theta$ internal neighbors, more than the leaf's $64R - 1$ possible neighbors. Label each nonempty final block by one of its own vertices, so shaved singleton labels cannot collide with a retained-block label.

Lemma 27 (Preservation and recoverable size)**.** *The resulting partition respects every friendly shore of value at most $2N$. Its number m_{out} of interblock edges satisfies*

$$m_{\mathrm{out}} \le 49B_0 + 60N\theta = O(Nb\log(2N) + NRt^2). \tag{56}$$

All these edges, with their original endpoints, can be learned in $O((N + m_{\mathrm{out}})\log(2N))$ queries.

Proof. Suppose a friendly shore S splits two retained vertices of a leaf W. A retained vertex on either side has at least

$$d(v)/6 - d_{\mathrm{out}}(v) \ge d(v)/8 \ge (15/2)\theta$$

neighbors on its own side within W. Simplicity forces both sides of $S \cap W$ to have more than θ vertices. But its induced cut has size at most $c_G(S) \le 2N$, contradicting Equation (54). Singleton blocks cannot be split.

Every interblock edge either joins distinct leaves or is incident to a shaved vertex. Vertices shaved for $d(v) < 60\theta$ account for fewer than $60N\theta$ incident edges. Every remaining shaved vertex

satisfies $d(v) < 24d_{\text{out}}(v)$, so its incident edges contribute at most $24\sum_v d_{\text{out}}(v) = 48B_0$. Adding the interleaf edges proves Equation (56). Overcounting an edge only makes the bound safer.

For each vertex, enumerate its neighbors outside its final block by Lemma 3. The sum of these row lengths is $2m_{\text{out}}$, so the stated query bound follows. Keep both endpoint labels for every learned edge. Knowing only the aggregated weight between blocks would not determine the individual internal degrees needed below. □

15.2 The corrected friendly-cut structural argument

For completeness we prove the exact structural fact needed from the friendly-cut method [20, Claim 3.3 and Lemma 3.4]. Minimum degree two is used only here; triangle padding below supplies it. This proof does not assume that nonterminal vertices have degree at least three. The proof of [20, Lemma 3.4] asserts that every vertex of X has degree at least three; a degree-two vertex of X with both neighbors in S is consistent with minimality, and the argument below covers it.

Lemma 28 (Structured terminal minimum)**.** *In a simple unweighted graph of minimum degree at least two, every pair of distinct terminals has a minimum shore S, oriented toward one of the terminals r, such that either S is friendly or $S \setminus \{r\}$ is friendly. The relevant friendly shore has value at most $2N$.*

Proof. Take any minimum terminal cut. For a nonterminal vertex x, moving x alone to the other side remains a terminal cut. Optimality gives

$$2\,\mathrm{cr}_S(x) \le d(x). \tag{57}$$

Hence an unfriendly vertex must be a terminal. If the cut is friendly, its value is at most either terminal degree, hence less than N. Otherwise first choose an unfriendly terminal r, orient its shore to contain r, and shrink that shore to an inclusion-minimal minimum r-shore. Shrinking can only increase r's crossing degree, so r remains unfriendly. Call the resulting shore S, and let u be the other terminal. Put $X = S \setminus \{r\}$.

For $x \in X$, removing x from S preserves separation and strictly increases its cut value by inclusion-minimality. Thus $2\,\mathrm{cr}_S(x) < d(x)$, and integrality gives $2\,\mathrm{cr}_S(x) + 1 \le d(x)$. Moving r increases x's crossing degree by at most one, by simplicity, so

$$\mathrm{cr}_X(x) \le (d(x) + 1)/2 \le 5d(x)/6$$

when $d(x) \ge 2$. In particular a degree-two such vertex has $\mathrm{cr}_S(x) = 0$ and $\mathrm{cr}_X(x) \le 1$. At r itself, the new crossing degree is $d(r) - \mathrm{cr}_S(r) < d(r)/6$. For an outside nonterminal, deleting r can only decrease crossing degree, so Equation (57) suffices.

It remains to check u. Both terminals cannot be unfriendly in a minimum cut. Indeed their crossing stars overlap in at most the edge ru, whence, writing $\lambda = c_G(S)$,

$$\mathrm{cr}_S(r) + \mathrm{cr}_S(u) \le \lambda + 1, \qquad \lambda \le \min\{d(r), d(u)\}.$$

If both were unfriendly and $d(r) \le d(u)$, their crossing sum would exceed $5(d(r) + d(u))/6 \ge 5d(r)/3 > d(r) + 1$, since $d(r) \ge 2$. This contradicts the displayed upper bound. Therefore u is friendly in S, and its crossing degree only decreases in X. We have proved friendliness of X at every vertex. Finally

$$c_G(X) = c_G(S) + d(r) - 2\,\mathrm{cr}_S(r) \le 2d(r) < 2N.$$

The order of choosing r and minimizing its shore is essential: complementing an inclusion-minimal shore would not preserve minimality on the other side. □

15.3 Exact star representation and recovery

Let $\mathcal{P}$ be the final partition. All vertex degrees and all interblock edges are known. For a block B and $v \in B$, compute

$$d_B(v) = d_G(v) - |E_G(v, V \setminus B)|.$$

The following is the star transform of Kenneth-Mordoch and Krauthgamer [20, Claim 3.5], applied to our blocks. Construct an explicit graph H with the original vertices and a new proxy p_B for each block. Keep every interblock edge with its original endpoints, remove all unknown internal edges, and add the edge vp_B of capacity $d_B(v)$ for every $v \in B$. Zero capacities may be omitted. There are at most $2N$ vertices and $m_{\text{out}} + N$ nonzero edges.

Lemma 29 (Exact values and every returned shore)**.** *For every distinct pair s, t of original vertices, $\lambda_H(s,t) = \lambda_G(s,t)$. Every minimum s–t shore A in H restricts to a minimum shore $A \cap V(G)$ in G, and $c_H(A) = c_G(A \cap V(G))$.*

Proof. For an arbitrary A, let $S = A \cap V(G)$. The contribution of the star of B is either $\sum_{v \in S \cap B} d_B(v)$ or $\sum_{v \in B \setminus S} d_B(v)$, depending on its proxy's side. Each sum counts every original internal edge crossing S once and possibly counts additional internal incidences. Interblock contributions are exact. Therefore

$$c_H(A) \geq c_G(S). \tag{58}$$

By Lemmas 27 and 28, the pair has a minimum shore which either respects all blocks or is obtained by adding one terminal r to a shore X respecting all blocks. In the first case place every proxy with its block; all star contributions and original internal cut contributions are zero. In the second case place every proxy with the side assigned to its block by X, and then add the original vertex r to the shore. Only r's block is split, and it is split by isolating r within that block. Its star cost $d_B(r)$ is exactly the original internal crossing cost. Interblock edges again have their exact original contribution. In either case the original minimum has an extension of the same value to H. If the chosen orientation contains t, complement both shores to obtain an s-shore.

Every separating shore in H restricts to one in G. Applying Equation (58) gives $\lambda_H \geq \lambda_G$, and the exact extension gives the reverse inequality. If A is any minimum shore in H, then

$$\lambda_G \leq c_G(A \cap V(G)) \leq c_H(A) = \lambda_H = \lambda_G.$$

All inequalities are equalities. This proves both optimality and the exact original value of every returned restricted shore. □

Enumerating shores of the explicit H now yields the requested answer for every pair with no input queries. One may instead use any exact explicit-graph minimum-cut routine. The argument is simultaneous for all pairs; it does not rerun the oracle construction separately for them. A flow through a proxy may have no corresponding original routing, so Lemma 29 is deliberately a value-and-shore statement.

15.4 Padding, total query caps, and the final exponent

Proof of Theorem 2. For each original vertex v, attach two new vertices v', v'' and the three known edges of the triangle $vv'v''v$. The padded graph $\widehat{G}$ is simple, has $N = 3n$ vertices, and has minimum degree at least two. A query on a padded shore is one query on its original-vertex intersection plus a known triangle contribution. Every original shore extends with all three vertices of each triangle on the same side at no additional cost. Conversely every padded shore costs at least its original

restriction. The same two inequalities as in Lemma 29 show that any padded minimum between originals restricts to an exact original minimum with the same value. Thus padding transports both outputs, not just the optimum number.

Apply the complete construction to $\widehat{G}$. Its global parameters satisfy

$$t = \lceil N^{1/3} \rceil \le 2N^{1/3}, \qquad b = 2t, \qquad R = O(\log(2Nb)) = O(\log(2N)).$$

By Lemma 26, decomposition costs

$$O(Nt^2R^3 \log^2(2N)) = O(N^{5/3} \log^5(2N)).$$

Degree measurements and Lemma 27 cost

$$O\Big((N + Nb \log(2N) + NRt^2) \log(2N)\Big) = O(N^{5/3} \log^2(2N)).$$

All star construction and all-pairs optimization are offline. Since $N = 3n$, these bounds give the stated exponent and logarithmic power. The returned auxiliary minimum restricts first to the padded graph and then to the original graph; both restriction steps preserve its exact cut value. Complement if necessary to output the prescribed orientation.

To make the cap apply to every reply history, fix a sufficiently large integer upper bound for the displayed costs before querying. Every procedure has deterministic tie-breaking; its public state consists only of known parameters, learned edges, explicit flows, matching records, and replies. Bound each binary search by its public set size, each finite graph search by exhaustive enumeration, each phase by its unit source-demand fuel, and the recursive execution by a public finite cardinality/depth fuel. Reject invalid counts or infeasible public updates, and terminate with a fixed default answer if any fuel or the global query cap is exhausted. One can equivalently unfold this guarded procedure into a finite adaptive query tree. Every local computation terminates on arbitrary replies; the guard is checked before each query. This also covers inconsistent answers to repeated queries.

On an honest graph, the residual, legal-response, partition, and recovery invariants proved above hold at every stage, and the bounds pay for all queries, including failed games, final unsuccessful searches, and degree measurements. Thus no guard truncates an honest execution, and no recovery failure or default answer is reached. The uniform capped tree therefore has the asserted correctness on every promised input. For $n \le 1$ there is no distinct terminal pair; a separately defined trivial procedure may return an empty answer family. This harmless convention is not needed for the stated $n \ge 2$ bound or for the formal endpoint's singleton budget. □

15.5 Relation to the formal endpoint

The proof above is mathematical and stands independently of Lean. In the Lean library's `DeterministicAPMCExpansion.Pruning` namespace, the declarations `program`, `budget`, and `deterministic_allPa` choose the program and its cap before n's hidden graph. Their composition executes decomposition, shaving, endpoint recovery, star construction, and triangle unpadding. A literal restatement in the library's endpoint checks also fixes the constants and checks arbitrary full-history-dependent replies.

The formal arithmetic uses $\ell(n) = \lceil \log_2(2n + 2) \rceil + 1$ and proves $\mathrm{budget}(n) \le Cn^{5/3}\ell(n)^5$ for $n \ge 2$, with an explicit positive constant. It also proves a natural-log form and $o(n^2)$. Its construction uses the same four fresh full cut-player runs, actual flow responses, projected fake graph, and recursive partition invariants described above. Its shaving interface is somewhat stronger: it passes an additive partition with parameters $(2t, 8R, t^2)$, yielding the common scale $32Rt^2$ used in Equation (53).

The single-pair canonical-witness endpoint is formalized separately in the `CanonicalST` modules; Appendix A gives the current correspondence and its verification scope. The all-pairs theorem does not supply original flows from the recovered star graph and does not assume the randomized Gomory–Hu reduction [28].

AI use

The results in this paper were obtained by prompting Codex GPT-6 Astra (OpenAI), which also wrote this preliminary draft. The Lean proofs (Appendix A) were developed by Claude Opus 5.5 (Anthropic) and audited by GPT-6 Astra.

References

[1] Amir Abboud, Robert Krauthgamer, Jason Li, Debmalya Panigrahi, Thatchaphol Saranurak, and Ohad Trabelsi. Breaking the cubic barrier for all-pairs max-flow: Gomory–Hu tree in nearly quadratic time. In *63rd IEEE Annual Symposium on Foundations of Computer Science (FOCS 2022)*, pages 884–895, 2022. arXiv:2111.04958.

[2] Amir Abboud, Robert Krauthgamer, and Ohad Trabelsi. Friendly cut sparsifiers and faster Gomory–Hu trees. In *Proceedings of the 2022 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2022)*, pages 3630–3649, 2022. arXiv:2110.15891.

[3] Amir Abboud, Rasmus Kyng, Jason Li, Debmalya Panigrahi, Maximilian Probst Gutenberg, Thatchaphol Saranurak, Weixuan Yuan, and Wuwei Yuan. Deterministic almost-linear-time Gomory–Hu trees. In *66th IEEE Annual Symposium on Foundations of Computer Science (FOCS 2025)*, pages 659–666, 2025. arXiv:2507.20354.

[4] Aditya Anand, Thatchaphol Saranurak, and Yunfan Wang. Deterministic edge connectivity and max flow using subquadratic cut queries. arXiv:2410.18704v1, 2024. SODA 2025, pages 124–142.

[5] Simon Apers, Yuval Efron, Paweł Gawrychowski, Troy Lee, Sagnik Mukhopadhyay, and Danupon Nanongkai. Cut query algorithms with star contraction. In *63rd IEEE Annual Symposium on Foundations of Computer Science (FOCS 2022)*, 2022. arXiv:2201.05674.

[6] Deeparnab Chakrabarty, Andrei Graur, Haotian Jiang, and Aaron Sidford. Improved lower bounds for submodular function minimization. In *63rd IEEE Annual Symposium on Foundations of Computer Science (FOCS 2022)*, 2022. arXiv:2207.04342.

[7] E. A. Dinic. Algorithm for solution of a problem of maximum flow in networks with power estimation. *Soviet Mathematics Doklady*, 11:1277–1280, 1970.

[8] Shimon Even and Robert Endre Tarjan. Network flow and testing graph connectivity. *SIAM Journal on Computing*, 4(4):507–518, 1975.

[9] L. R. Ford, Jr. and D. R. Fulkerson. Maximal flow through a network. *Canadian Journal of Mathematics*, 8:399–404, 1956.

[10] Zvi Galil and Xiangdong Yu. Short length versions of Menger's theorem. In *Proceedings of the 27th Annual ACM Symposium on Theory of Computing (STOC 1995)*, pages 499–508, 1995.

[11] Ralph E. Gomory and Tien Chung Hu. Multi-terminal network flows. *Journal of the Society for Industrial and Applied Mathematics*, 9(4):551–570, 1961.

[12] Andrei Graur, Tristan Pollner, Vidhya Ramaswamy, and S. Matthew Weinberg. New query lower bounds for submodular function minimization. In *11th Innovations in Theoretical Computer Science Conference (ITCS 2020)*, 2020. arXiv:1911.06889.

[13] Haotian Jiang. Minimizing convex functions with integral minimizers. In *Proceedings of the 2021 ACM-SIAM Symposium on Discrete Algorithms (SODA 2021)*, 2021. Journal version: *Minimizing Convex Functions with Rational Minimizers*, Journal of the ACM 70(1). arXiv:2007.01445.

[14] Yonggang Jiang, Danupon Nanongkai, and Pachara Sawettamalya. Minimum s–t cuts with fewer cut queries. arXiv:2510.18274v1, 2025. Version 1, 21 October 2025; SODA 2026, pages 258–296.

[15] Tomáš Kaiser. A short proof of the tree-packing theorem. *Discrete Mathematics*, 312(10):1689–1691, 2012.

[16] David R. Karger. Minimum cuts in near-linear time. *Journal of the ACM*, 47(1):46–76, 2000.

[17] A. V. Karzanov. On finding maximum flows in networks with special structure and some applications. *Matematicheskie Voprosy Upravleniya Proizvodstvom*, 5:81–94, 1973. Moscow State University Press. In Russian.

[18] Yotam Kenneth-Mordoch. Deterministic global minimum cut using $\widetilde{O}(n^{3/2})$ cut queries. arXiv:2610.23461v1, 2026. Version 1, 16 October 2026; accepted at SOSA 2027.

[19] Yotam Kenneth-Mordoch and Robert Krauthgamer. All-pairs minimum cut using $\widetilde{O}(n^{7/4})$ cut queries. In *Proceedings of the 2026 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2026)*, pages 4077–4095, 2026. arXiv:2510.16741.

[20] Yotam Kenneth-Mordoch and Robert Krauthgamer. Faster all-pairs minimum cut: Bypassing exact max-flow. arXiv:2511.10036v2, 2026. Version 2, 6 April 2026; STOC 2026.

[21] Rohit Khandekar, Subhash A. Khot, Lorenzo Orecchia, and Nisheeth K. Vishnoi. On a cut-matching game for the sparsest cut problem. Technical Report UCB/EECS-2007-177, EECS Department, University of California, Berkeley, December 2007. Technical report.

[22] Rohit Khandekar, Satish Rao, and Umesh Vazirani. Graph partitioning using single commodity flows. *Journal of the ACM*, 56(4), 2009. Preliminary version in STOC 2006.

[23] Troy Lee. Finding and certifying weighted minimum cuts with cut queries. Companion manuscript. `https://github.com/troyjlee/cut-query-complexity/blob/main/papers/P1/finding-certifying-weighted-minimum-cuts.pdf`, 2026.

[24] Troy Lee, Tongyang Li, Miklos Santha, and Shengyu Zhang. On the cut dimension of a graph. In *36th Computational Complexity Conference (CCC 2021)*, 2021. arXiv:2011.05085.

[25] Yin Tat Lee, Aaron Sidford, and Sam Chiu-wai Wong. A faster cutting plane method and its implications for combinatorial and convex optimization. In *56th IEEE Annual Symposium on Foundations of Computer Science (FOCS 2015)*, 2015. arXiv:1508.04874.

[26] Sagnik Mukhopadhyay and Danupon Nanongkai. Weighted min-cut: Sequential, cut-query, and streaming algorithms. In *Proceedings of the 52nd Annual ACM Symposium on Theory of Computing (STOC 2020)*, 2020. arXiv:1911.01651.

[27] C. St. J. A. Nash-Williams. Edge-disjoint spanning trees of finite graphs. *Journal of the London Mathematical Society*, s1-36(1):445–450, 1961.

[28] Maximilian Probst Gutenberg, Rasmus Kyng, Weixuan Yuan, and Wuwei Yuan. A simple and fast reduction from Gomory–Hu trees to polylog maxflows. arXiv:2509.02520, 2025. SODA 2026, pages 3805–3819; the proceedings version is marked retracted, and arXiv version 2 reports a gap in the proof of the running-time bound.

[29] Aviad Rubinstein, Tselil Schramm, and S. Matthew Weinberg. Computing exact minimum cuts without knowing the graph. In *9th Innovations in Theoretical Computer Science Conference (ITCS 2018)*, volume 94 of *Leibniz International Proceedings in Informatics*, pages 39:1–39:16. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2018.

[30] Thatchaphol Saranurak and Di Wang. Expander decomposition and pruning: Faster, stronger, and simpler. In *Proceedings of the 30th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2019)*, pages 2616–2635, 2019. arXiv:1812.08958.

[31] W. T. Tutte. On the problem of decomposing a graph into n connected factors. *Journal of the London Mathematical Society*, s1-36(1):221–230, 1961.

A Correspondence with the Lean formalization

This appendix maps the paper's definitions and all 29 numbered statements to the Lean library https://github.com/troyjlee/cut-query-complexity, version 0.1.0. The module `CanonicalST/SkeletonCounterexample.lean` formalizes Proposition 4. The correspondence concerns the statements of the declarations, rather than an assertion that every informal proof step is reproduced in the same order. In particular, the single-pair theorem has its own formal endpoint, independently of the all-pairs theorem.

To keep the tables readable, declaration names use the following namespace abbreviations. Unprefixed project names are relative to `CutQueryComplexity`; all four expanded namespaces below are also relative to that root.

Prefix	Namespace
`C.`	`CanonicalST.`
`E.`	`DeterministicAPMCExpansion.`
`A.`	`AllPairsAudit.`
`P.`	`AllPairs.`

The source files for these groups lie below the corresponding directories of `CutQueryComplexity/`.

A.1 Model and definitions

Lean represents the simple input as a loopless symmetric nonnegative `WeightedGraph` V, together with `A.Unweighted`, which requires every edge weight to be zero or one. The vertex type is finite; the all-pairs endpoint uses Lean's type `Fin` n. Real-valued oracle answers allow the same program type to describe inconsistent replies, but answers on the promised inputs are the exact integer cut values of the paper. The type `DTree` records a public upper bound on the number of adaptive queries, and `DTree.cost_le` applies to every answer function. The program is chosen before the hidden graph.

Paper notation or definition	Lean counterpart
$c_G(S)$ and a terminal minimum	`WeightedGraph.cut`; `IsSTMinCut`; `C.Augmentation.terminalValue`. The predicate `IsSTMinCut` $G\,s\,t\,S$ uses $s \in S$, $t \notin S$. The canonical witness family uses the paper's opposite orientation; `C.canonicalFamily_contains_output_optimum` complements it.
Integral flow and single-pair output	`C.Augmentation.FlowSpec`; `C.Output`; `C.OutputSpec`. The flow is an antisymmetric function on ordered vertex pairs, with integral values, edge-capacity constraints, and the prescribed divergences. The output contains the shore, value, and flow.
Core C and learned graphs K, J	`C.CoreData`; `C.CoreSpec`; `C.Phase.Invariant`. The specification records exact outside-core edges, the minimum degree of J, the size bounds, and containment in the input graph.
Fixed extension S_x and family $\mathcal{A}$	`C.AdmissibleTrace`; `C.canonicalExtension`; `C.canonicalFamily`. These depend on C, K, s, t, independently of later learned core edges.
Potential and witness multiplicity	`C.Witness.potential`; `C.Witness.multiplicity`; `C.Witness.potential_eq_weighted_sum`. The potential is rational and averages over padded witnesses, with exactly the multiplicities in Equations (18) and (19).

Paper notation or definition	Lean counterpart
Friendly shore and preserving partition	`A.Friendly`; `A.PreservesFriendly`. Friendliness is the division-free condition that at most five sixths of a vertex's degree crosses the shore. Preservation applies simultaneously to friendly shores of value at most $2\|V\|$.
Auxiliary star and all-pairs output	`A.star`; `A.originalShore`; `P.Answers`; `P.allPairsGood`. The star has vertex type $V \oplus B$, where B indexes blocks. Correctness concerns original shores and values for distinct terminal pairs; it does not assert original maximum-flow recovery.

A.2 Numbered statements

Each row includes all assertions of the indicated statement, or specifies the limitation of the listed counterpart. Several statements are proved by combining a correctness declaration with a separate budget declaration. In the all-pairs construction, Lean records an additive partition with parameters $(2t, 8R, t^2)$; its shaving scale is $32Rt^2$, the same θ as in Equation (53).

Paper	Lean declarations	Correspondence

Theorem 1
Terminal cuts
	`C.exists_terminal_sqrt_program`
	`C.headlineBudget`
	`C.OutputSpec`
		One graph-independent program returns an original minimum shore, its value, and an integral maximum flow. Its cap is $\lceil 2^{23} n\sqrt{n}\,\lceil \log_2(2n)\rceil^3\rceil$ for $n \geq 2$, including arbitrary replies.

Theorem 2
All pairs
	`E.Pruning.deterministic_allPairs`
	`E.Pruning.program_exact`
	`E.Pruning.program_cost_bound`
		Exact original shores and values for every distinct ordered pair. The public cap is $Cn^{5/3}(\lceil \log_2(2n+2)\rceil + 1)^5$. The endpoint also proves $o(n^2)$.

Lemma 3
Finding an edge
	`crossQuery_run`
	`findCross_logarithmic`
	`C.rowPrefix_complete`
	`C.rowPrefix_cost`
		The rectangle count costs three queries; crossing-edge search has cap $6(\lceil \log_2 n\rceil + 1)$. A row with d neighbors is recovered with fuel d once its count is known.

Proposition 4 Weak-skeleton example	`C.exists_weakSkeleton_terminal_gap`	The literal four-vertex construction for every real $M \geq 1$: G is unweighted, B has support contained in G, both global minima and both singleton cuts equal one, and the terminal values are two and M.
Lemma 5 Preprocessing	`C.coreProgram_spec` `C.CoreSpec`	The actual peeling and seed-edge program satisfies all three structural clauses. Its cap is $n + 6nD(\lceil \log_2 n\rceil + 1)$.
Lemma 6 Fixed canonical cuts	`C.canonicalFamily_contains_optimum` `C.coreProgram_canonical_optimum`	The same fixed family contains an optimum for every graph $K + L$ supported on the core, and for the original input after preprocessing.
Lemma 7 Small-cut count	`C.Counting.smallCutLabelings_card_le`	Both orientations are counted, without a connectivity hypothesis. The explicit bound is $(2n)^{\lceil 32(N+b)/D\rceil}$.
Corollary 8 Canonical-family count	`C.Counting.canonicalFamily_ambientPower_card_le`	The low-cut canonical family has size at most $(2n)^{\lceil 64n/D\rceil}$, uniformly in every later learned core containing J.

Paper	Lean declarations	Correspondence

Definition 9
Padded witness
	`C.Witness.possiblePairs`
	`C.Witness.paddingFamily`
	`C.Witness.canonicalWitnesses`
	`C.Witness.mem_paddingFamily`
		Set membership is precisely the padding condition. Each crossing pair is represented once, with its endpoint in the shore first; padding may include nonedges.

Lemma 10
Witness invariants and count
	`C.Witness.canonicalWitnesses_empty_iff`
	`C.Witness.paddedWitnesses_addEdge`
	`C.Witness.phase_canonicalWitnesses_card_le`
		Respectively: the threshold test, exact removal without creation, and the bound $(2n)^{\lceil 64n/D\rceil+2k}$ on the number of witnesses.

Lemma 11 Witness loss	`C.Witness.phase_witness_loss` `C.Witness.witness_loss_card_identity`	The loss inequality is stated after multiplying by $k\|\mathcal{W}\|$. Dividing by the positive witness count gives the paper's probability bound.
Lemma 12 Acyclic support	`C.Augmentation.supportSize_le` `C.Augmentation.cycle_free_support_bound`	The literal $\sqrt{2}\,n\sqrt{r}$ bound; the first declaration uses a topological numbering and the second obtains it from acyclicity.
Lemma 13 Sparse augmentation	`C.Augmentation.exists_sparse_augmentation`	An unknown unit subgraph Q has at most $\sqrt{2}\,n\sqrt{k}$ edges and supports a value-f flow together with the learned graph. This is an existence theorem, not an edge-discovery procedure.
Lemma 14 Long unknown edge	`C.Augmentation.exists_long_unknown_edge` `C.Augmentation.phaseDelta_ge_sqrt`	The exact rational threshold is $(\lfloor k/2\rfloor+1)/(2n\lceil\sqrt{k}\rceil)$, with the stated $\sqrt{k}/(8n)$ lower bound.

Lemma 15
Long-edge search
	`C.LongEdge.longEdge_correct`
	`C.LongEdge.longEdge_cost`
	`C.LongEdge.longEdgeBudget_le`
		A returned edge has length at least $\delta/2$; failure excludes every edge of length at least δ. The cap is $72\lceil\log_2\lceil 1/\delta\rceil\rceil+6\lceil\log_2 n\rceil+150$.

Theorem 16 Deficit reduction or refutation	`C.Phase.program_correct` `C.phaseQueryBudget_le`	Success or target refutation holds without a feasibility promise. The public phase cap is at most 2^{19} times the expression in Equation (28), using $\lceil\log_2(2n)\rceil$.
Lemma 17 Threshold decision	`C.Decision.program_correct` `C.decisionQueryBudget_le`	Geometrically decreasing deficits decide the target while retaining learned edges. The cap is at most $2^{21}(n^2/D+n\sqrt{f})\lceil\log_2(2n)\rceil^2$.
Lemma 18 Residual reachability	`C.Reachability.program_correct` `C.Reachability.program_cost`	The returned set is exactly the source-reachable set in the original residual graph, and its cut equals the flow value. Complementation gives the paper's t-oriented shore.

Theorem 19
Heavy terminal stars
	`E.Flow.scaledExactProg_correct`
	`E.Flow.scaledExactProg_cost_sharp`
	`E.Flow.exists_scaledExactTree`
		Exactness allows arbitrary known integral stars. The exported sharp cap is the $q\le t^3$, $C\le R$, $F\le qt$ specialization used here. The general optimized bound in the theorem's first display is not exported as a single declaration.

Paper	Lean declarations	Correspondence

Lemma 20
Separating assignments
	`C.LongEdge.family_length`
	`C.LongEdge.family_adjacent`
	`C.LongEdge.family_separates`
		The same combinatorial specification, with an explicit recursive family of size $8\lceil \log_2 \ell \rceil$. Lean uses this construction in place of the paper's probabilistic existence proof and exhaustive selection.

Lemma 21 Heavy-star residual bound	`E.Flow.scaled_residual_middle_layer_deficit` `E.Flow.scaled_internal_pair_bound`	The remaining flow value is at most $8Cq^2/d^2$ for $d \geq 4$. Only the internal pair-capacity bound is used; the terminal stars need no capacity bound.

Lemma 22
Deterministic cut player
	`E.CutMatching.cutPlayer_expands`
	`E.CutMatching.cutPlayer_cost`
	`E.CutMatching.normalizedRounds`
		For a nonempty even clone set and legal responses, four fresh full runs give unit expansion. The cap counts all sixteen balanced subgames and the four final requests.

Lemma 23 Executed matching round	`E.matchingRound_correct` `E.RoundCorrect`	The actual flow produces the balanced sparse split or a legal matching with routed real pairs and at most βb fake pairs. The routing is part of the returned data.

Lemma 24
Constructed fake graph
	`E.Pruning.ActualRounds.completed_certificate`
	`E.Pruning.FakeEdges.sum_degrees`
	`E.Pruning.FakeEdges.history_cut_split`
		The graph is formed from the executed rounds, keeping multiplicity and deleting projected loops. These declarations give the cut certificate, edge and degree caps, and total degree $2M$.

Lemma 25
One-flow pruning
	`E.Pruning.prune_size_boundary`
	`E.Pruning.prune_boundary_strict`
	`E.Pruning.prune_induced_expansion`
	`E.Pruning.pruneProg_structure`
		The first three declarations prove the general cut algebra, including $b\|P\| \leq 4M$. The last applies it to the executed flow with $b = 2t$, yielding the large induced expander.

Lemma 26
Depth, mass, and boundary
	`E.Decomposition.WorkTree.depth_le_clog`
	`E.Decomposition.WorkTree.mass_le_clog`
	`E.Decomposition.State.representativeMap_boundary`
	`E.Decomposition.run_cost_work`
	`E.Pruning.decomposition_budget_le`
		Depth and active mass include failed games. The boundary is recorded as a sum of block cuts, equal to $2B_0$. The executed recursive cost gives the stated $O(Nt^2R^3 \log^2(2N))$ bound.

Lemma 27
Preservation and recovery
	`E.shaveProgram_preservesFriendly_of_additive`
	`P.shaveProgram_boundary_le`
	`P.recoverCapped_correct`
	`P.recoverCapped_cost`
		Simultaneous friendliness preservation, the constants 49 and 60, and exact interblock recovery. Lean's boundary sum counts each interblock edge twice; dividing its inequality by two gives Equation (56).

Lemma 28
Structured minimum
	`A.exists_structured_minimum`
	`A.lemma34`
	`A.degree_two_crossing_zero`
		The minimum is oriented toward the chosen unfriendly terminal; that terminal is then removed. The corrected argument includes degree-two nonterminals and the $2N$ cutoff.

Paper	Lean declarations	Correspondence

Lemma 29
Exact star representation
	`A.star_cut_dominates`
	`A.exists_exact_minimum_lift`
	`A.star_minimum_extract`
		Cut domination and an exact minimum lift establish equality of terminal values. Every auxiliary minimum shore restricts to an original minimum with the same value.

The final composition in Sections 9 and 15 additionally uses `P.paddedQuery_eval` and `P.padded_minimum_extract` for triangle padding, and `P.recoverAnswers_correct` for the recovered star answers. Its public program is `E.Pruning.program`. The general heavy-star analysis behind Theorem 19 is represented by `E.Flow.scaledExactProg_history` and `E.Flow.PhaseHistory.scaled_phase_count`, `E.Flow.PhaseHistory.scaled_length_bound`; the sharp endpoint specializes these estimates to the common recursive parameters.

A.3 Verification scope

Every declaration named above was resolved with `#check` against a fresh build of version 0.1.0, and its dependencies were inspected with `#print axioms`. The reported axiom sets are contained in {`propext`, `Classical.choice`, `Quot.sound`}.